

Special Seminar on Multi-Modal AI

Jiayuan Mao

Department of CIS, GRASP

<https://cis7000-fall26.thelabone.org/>

Lecture 7: Controllable and Structured Generation

Special Seminar on Multi-Modal AI

Jiayuan Mao

Department of CIS, GRASP

<https://cis7000-fall26.thelabone.org/>



Reading Assignments

- **Reading assignments are important!**
- Please read the papers **BEFORE** coming to the class; or you will learn nothing
- Official grades started from Lecture 7 (today)
- Each student is allowed 3 absences and 2 late submissions
- No additional exceptions
- This accounts for 20% of your grade

Project Proposals

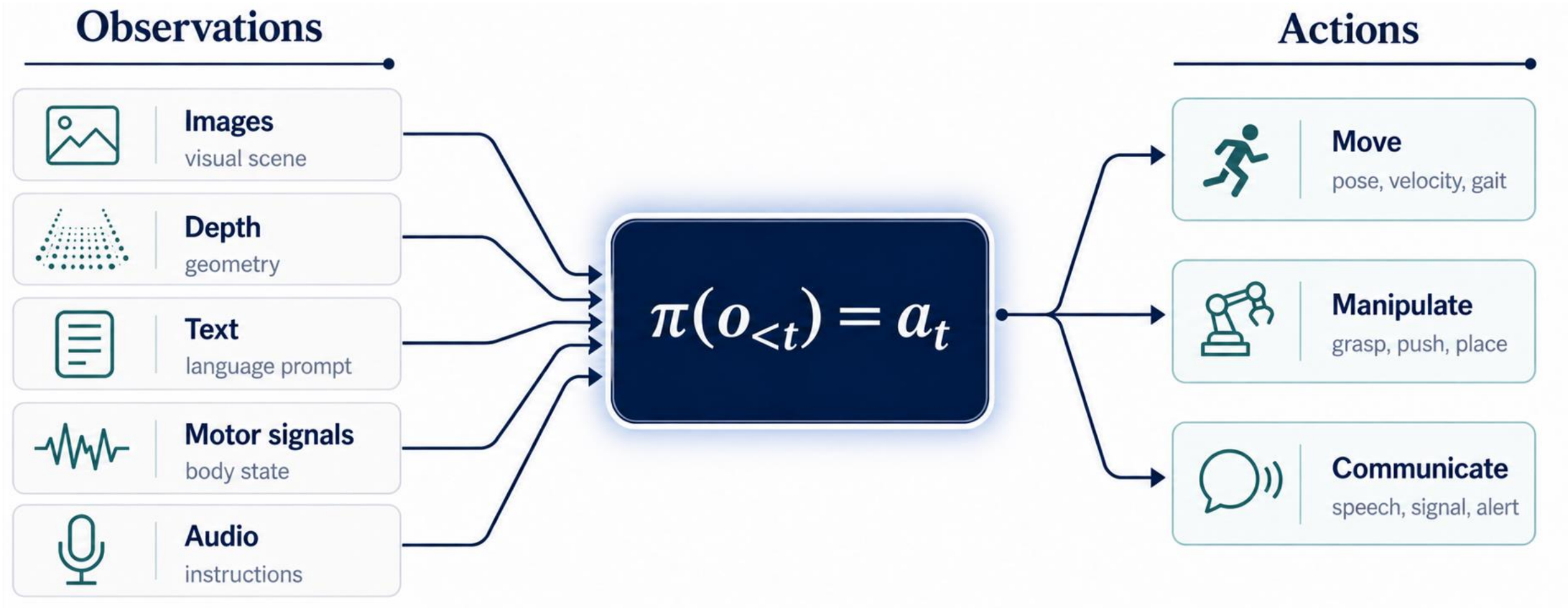
- All students have submitted their proposals!
- I will provide feedbacks to individual proposals in 2 weeks

Goal of This Course

Use Machine Learning to Build Intelligent Agents

Abstract View of An Agent: Multimodal AI

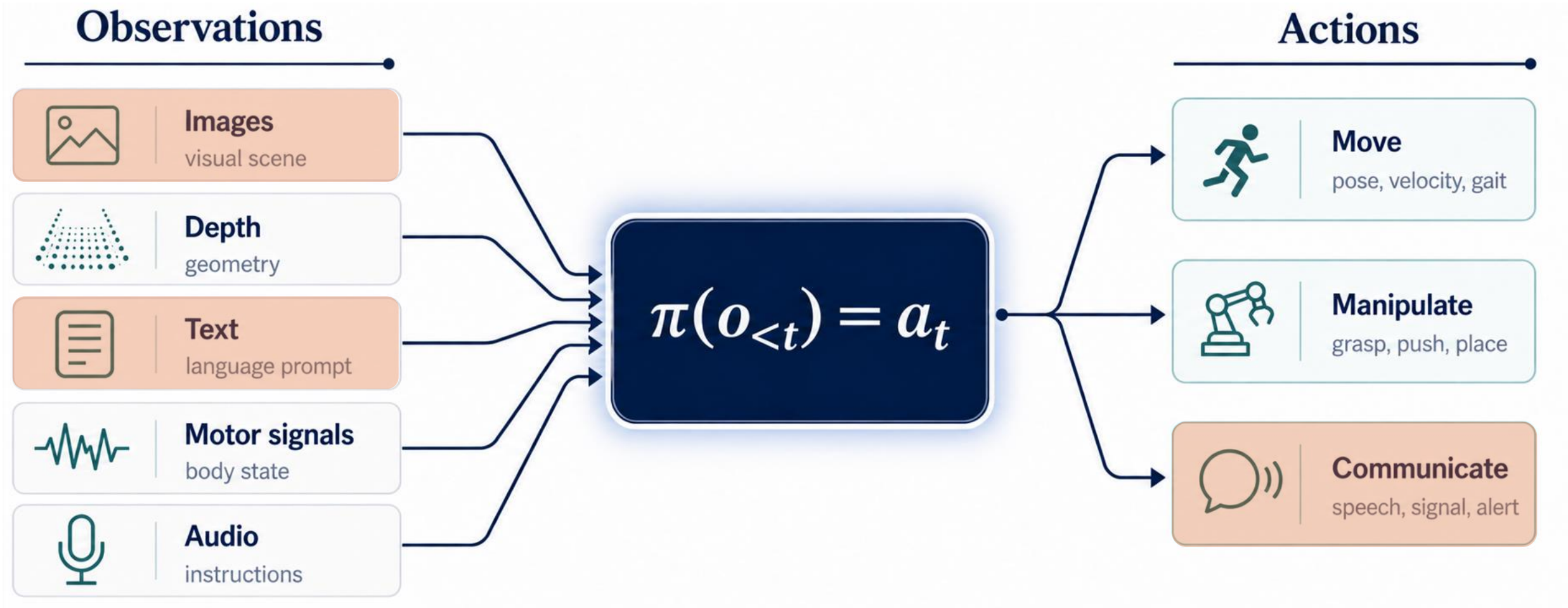
π : observations $\rightarrow$ action



Notation: t – time. $o_{<t}$ - all observations before t . a_t - action at t .

Abstract View of An Agent: Multimodal AI

π : observations $\rightarrow$ action



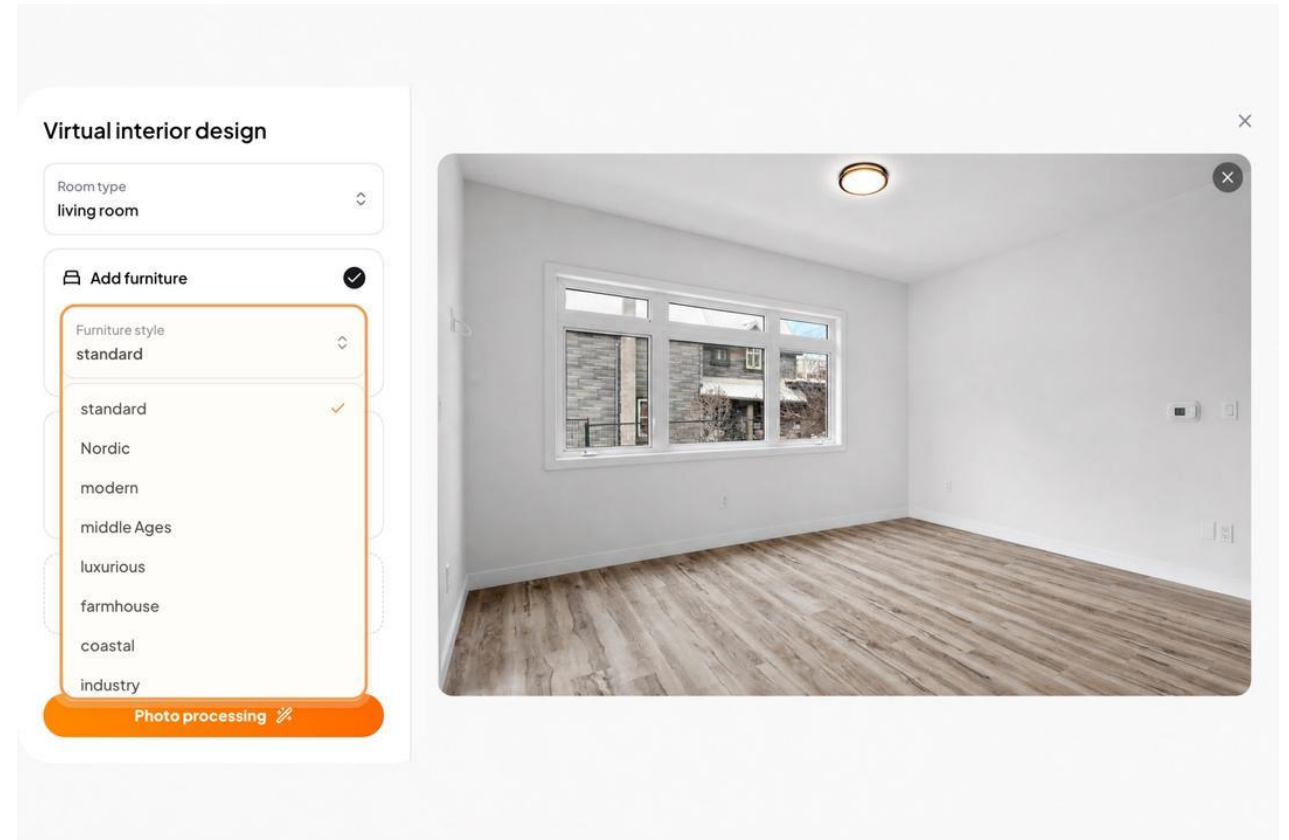
Notation: t – time. $o_{<t}$ - all observations before t . a_t - action at t .

Goal of This Lecture

Use machine learning to build a $p(\text{vision} \mid \text{controls})$

Edensign: Virtual Staging

- **Input:** A room photograph, room type, and preferred interior style
- **Controls:** Add or replace furniture, remove objects, and coordinate multiple views



Edensign: Content Creation with Control

Common interface: Text to Image

“A modern living room”



BLACK BOX



One finished image

Desired control: Specify the change

Material: Change only the sofa



Position: Move the table 30 cm

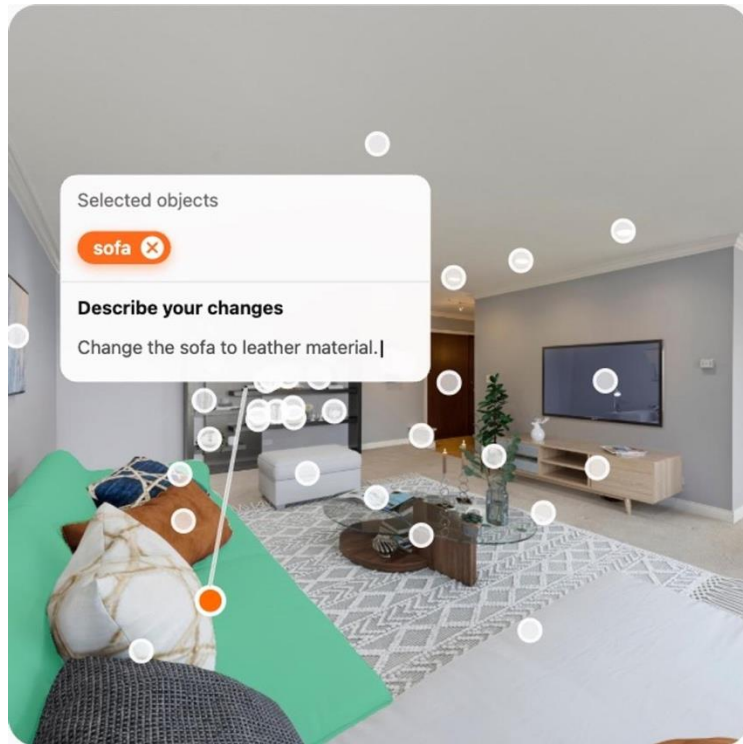


Viewpoint: Keep the same room



Local Editing

- **Goal:** Select the sofa and request a leather material
- **Control requirement:** Change the selected object while preserving the rest



Multi-View Control

- **Goal:** Consistent furniture placement across views of one room



Today's Agenda: **Ideas for Control**

1. Learning from examples

- Paired and unpaired label training

2. Leveraging neural network structures

- Patch structures and convolutional structures
- Compositional diffusion structures
- Neural network designs for editing tasks
- Graphics structures

3. Using programs to represent structures

Today's Agenda: Ideas for Control

1. Learning from examples

- Paired and unpaired label training

2. Leveraging neural network structures

- Patch structures and convolutional structures
- Compositional diffusion structures
- Neural network designs for editing tasks
- Graphics structures

3. Using programs to represent structures

Idea 1: Example-Based Learning or Finetuning

- **Training example:** source image + instruction + corresponding target image
- **Data bottleneck:** paired examples must demonstrate both the intended change and preservation

Idea 1.1: Learning Correspondence without Pairs

CycleGAN: $F(G(X)) = X; G(F(Y)) = Y$

Monet $\leftrightarrow$ Photos



Monet $\rightarrow$ photo

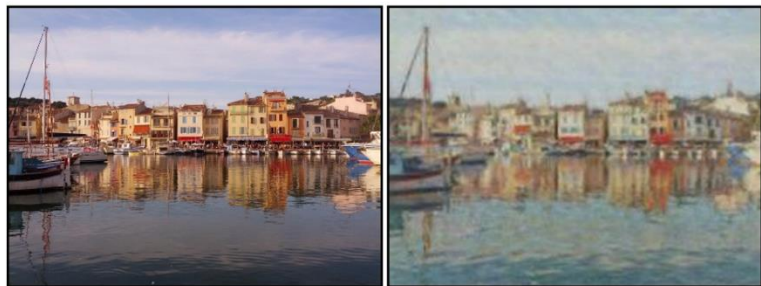


photo $\rightarrow$ Monet

Zebras $\leftrightarrow$ Horses



zebra $\rightarrow$ horse



horse $\rightarrow$ zebra

Summer $\leftrightarrow$ Winter



summer $\rightarrow$ winter



winter $\rightarrow$ summer



Photograph



Monet



Van Gogh



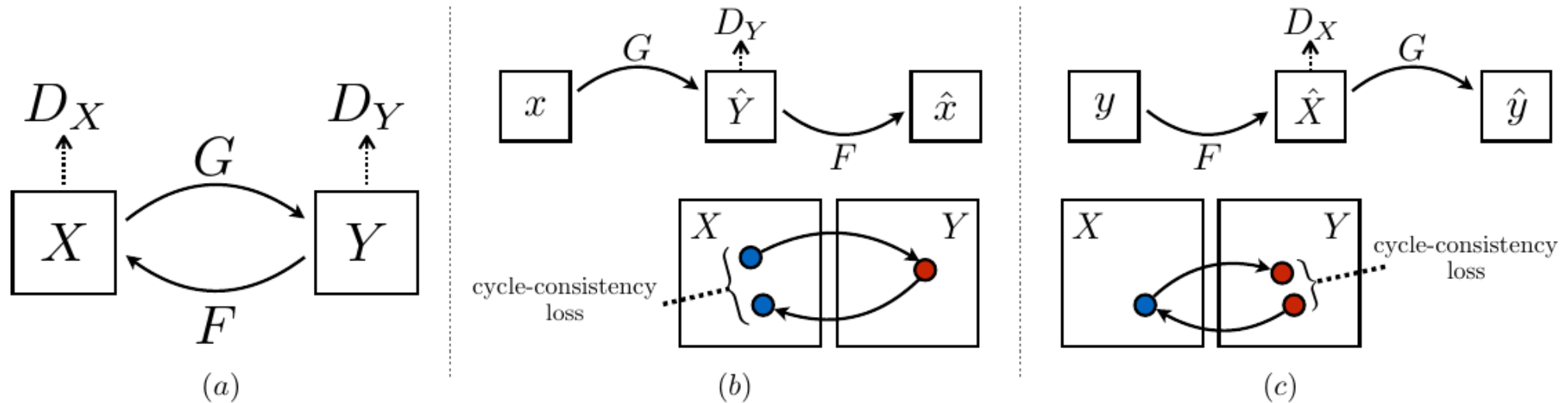
Cezanne



Ukiyo-e

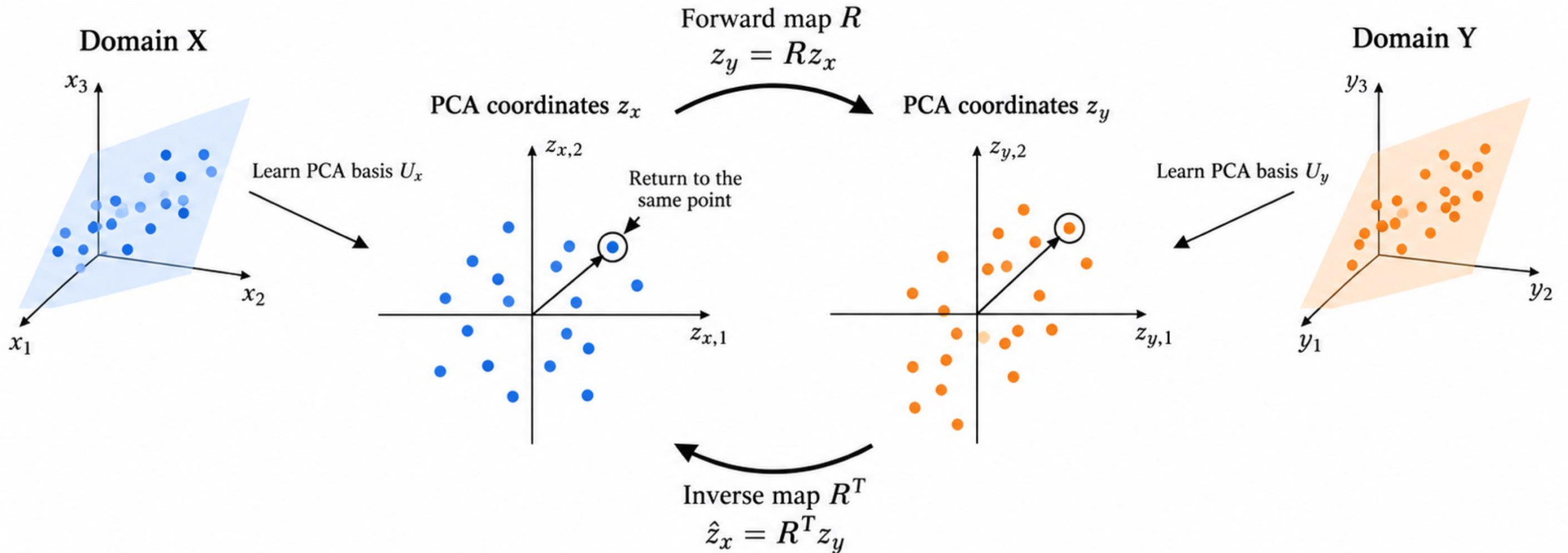
CycleGAN: $F(G(X)) = X; G(F(Y)) = Y$

- **Input:** Unpaired datasets of X and Y (e.g., paintings and photos)
- **Key idea:** Train forward and reverse mappings on unpaired images using adversarial and cycle reconstruction losses



CycleGAN: Principles and Limitations

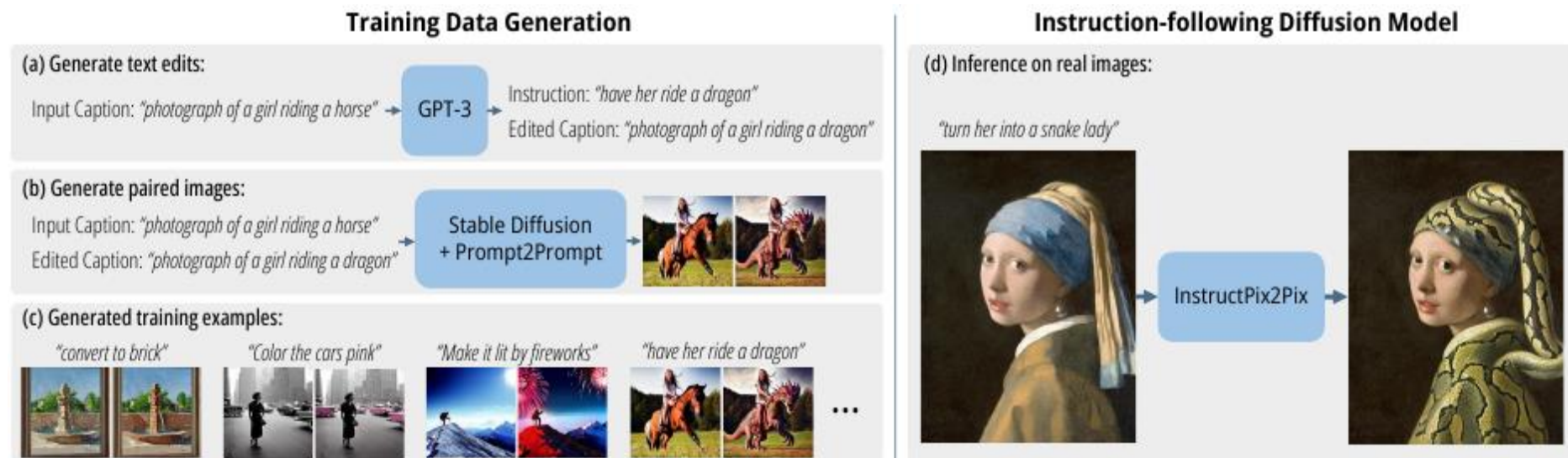
- **Principle:** Cycle reconstruction regularizes unpaired translation
- **Limitation:** A reversible mapping can still align the wrong semantics. For example, they still require two datasets to be “kind-of” aligned



Idea 1.2: Learning Correspondence with Pairs (through Texts)

InstructPix2Pix: Synthesize the Supervision

- **Key idea:** Synthesize instruction–before–after examples, then use them to fine-tune a single editor
- **Mechanism:** GPT-3 edits captions, image generation creates corresponding images, and CLIP filters poor pairs



The Limits of a Shared Seed

- Shared noise alone can still change object layout when the prompt changes
- InstructPix2Pix adapts attention sharing: self-attention is copied early in sampling
- Select from 100 candidate pairs per caption pair using CLIP-based filters



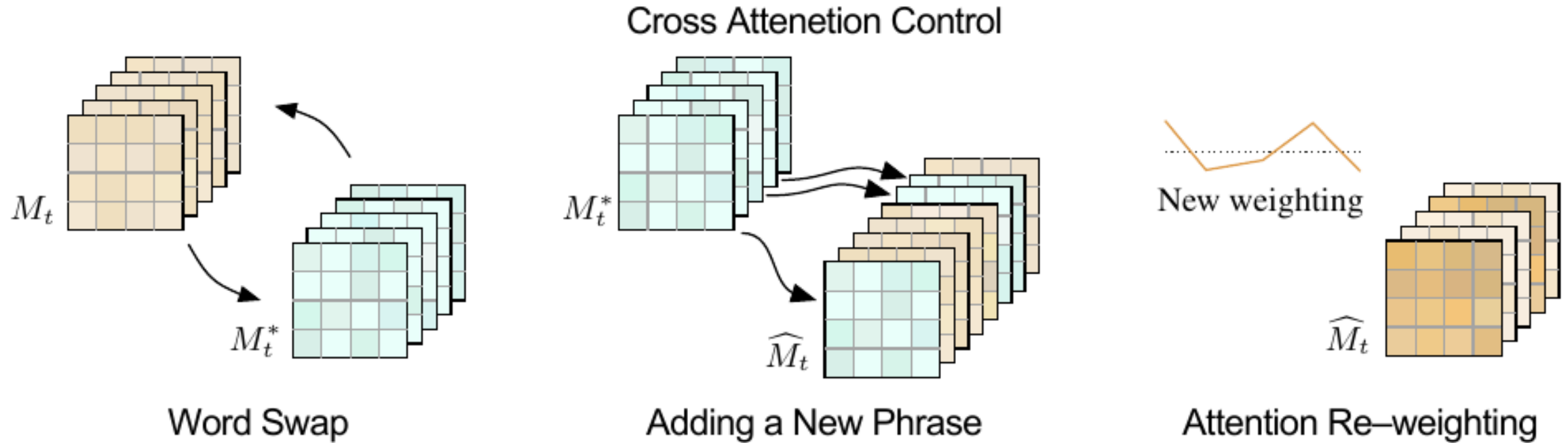
(a) Without Prompt-to-Prompt.

(b) With Prompt-to-Prompt.

Prompt-to-Prompt: Attention Control

- **Input:** source/edited prompts, corresponding tokens, and a coupled noise trajectory
- **Output:** an edited sample; selected source cross-attention maps retain layout

Prompt-to-Prompt: Reuse Attention Maps



- **Key idea:** Reuse source cross-attention maps to keep token–pixel layout while changing the prompt
- **Limitation:** It imposes a locality structure of the edits

Learning an Instruction Editor

- **Training input:** noisy target latent, encoded source image, and edit instruction
- **Prediction:** added noise

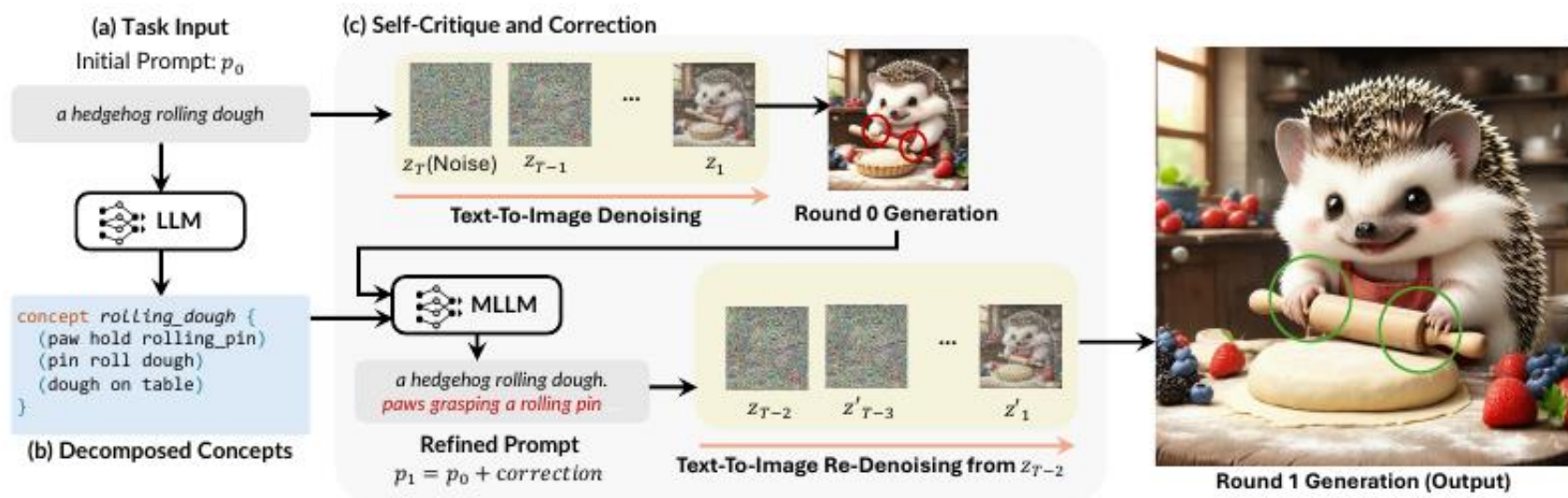
$$\mathcal{L} = \mathbb{E} \|\varepsilon - \varepsilon_{\theta}(z_t, t, E(c_I), c_T)\|^2$$

InstructPix2Pix: Principles and Limitations

- **Principle:** Synthetic instruction–before–after triples by editing in the language space
- **Limitation:** Edits can affect the wrong object or change too much; viewpoint and spatial rearrangements are difficult
- Failures in handling spatial relationships is a typical failure modes in generative models trained on general datasets
- Fundamentally related to the locality structures of image tokens and language tokens

DetailScribe: Synthesize the Edits

- **Task:** Generate high-fidelity images with complex object interaction prompts
- **Key idea:** After generating the first image, critique that with a VLM. Regenerate the image by updating the prompt
- **Mechanism for re-generation:** Partial re-denoising



Partial Re-denoising

- **Input:** the current image, a revised prompt, and a **selected noise level**
- Denoise from that specific noise level (not from x_1)
- Lower noise preserves more content; higher noise permits larger corrections

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Modern diffusion models usually have a nearly deterministic generation process after the very few steps (e.g., 3)

DetailScribe: Principles and Limitations

- **Principle:** Decompose interactions, critique specific failures, and partially re-denoise the existing image
- **Limitation:** Partial re-denoising struggles to add a missing main subject or repair a fundamentally wrong scene layout

Today's Agenda: Ideas for Control

1. Learning from examples

- Paired and unpaired label training

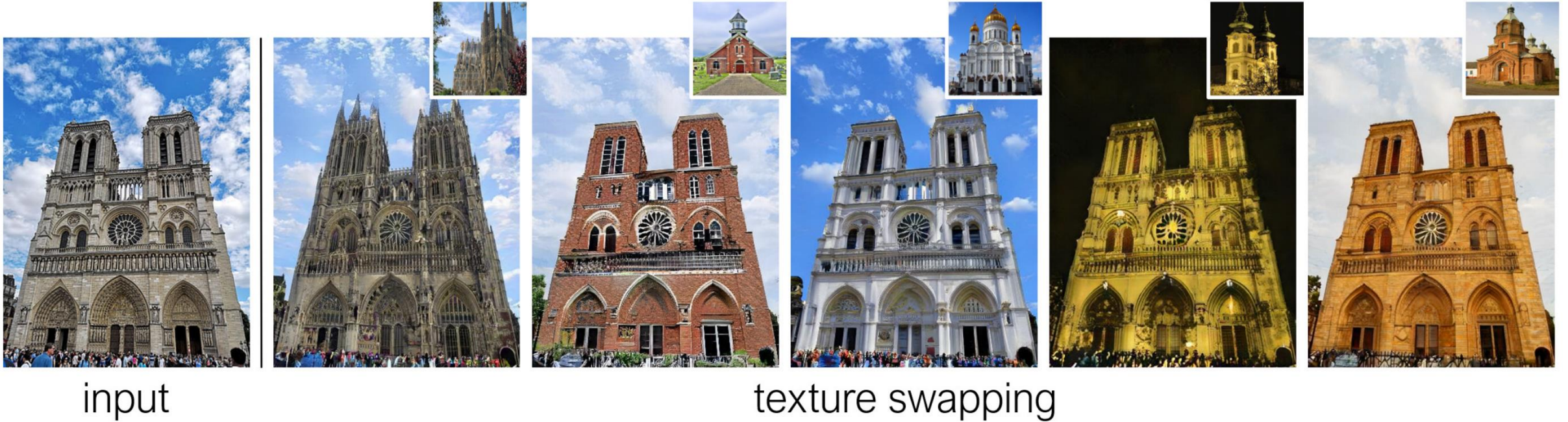
2. Leveraging neural network structures

- Patch structures and convolutional structures
- Compositional diffusion structures
- Neural network designs for editing tasks
- Graphics structures

3. Using programs to represent structures

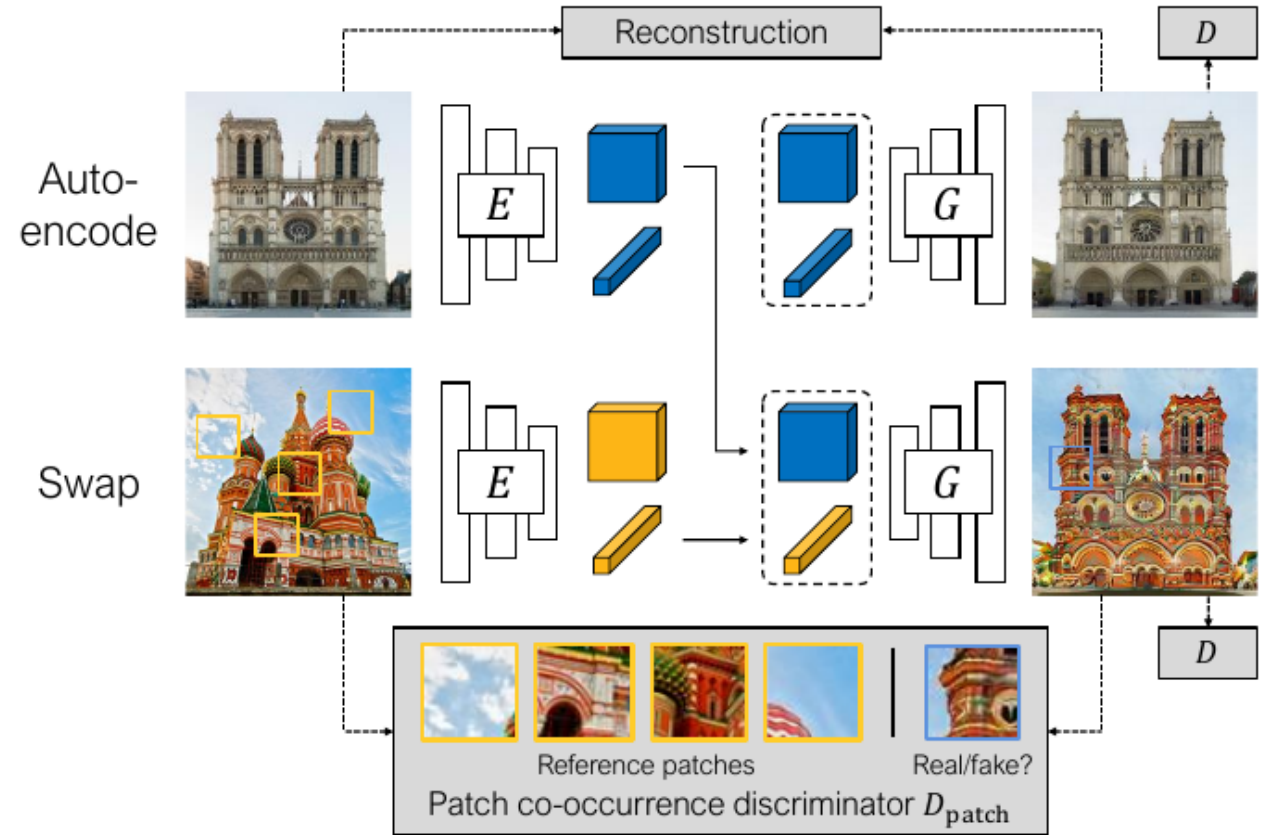
Idea 2.1: Patch Structures of Images

Swapping Autoencoder: Patch = Texture



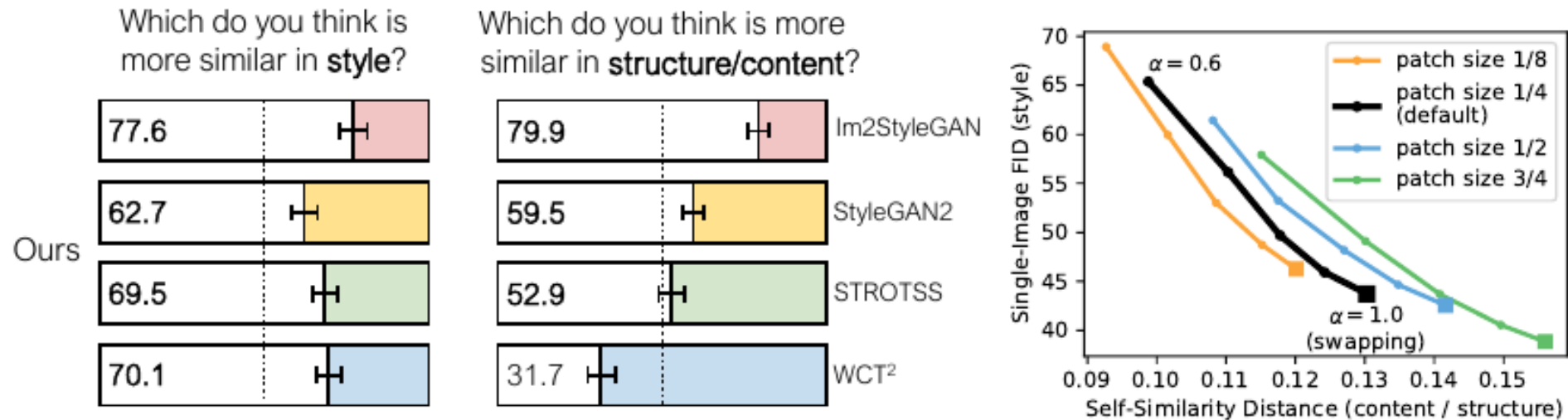
Swapping Autoencoder: Patch = Texture

- **Task:** Change B to change texture while retaining A's global structure
- **Model:** Use separate encoders and decoders for global structures and textures
- **Key idea:** Patch-level discriminator = texture-level discriminator



Texture Transfer versus Structure Preservation

- **Evidence:** Against StyleGAN2, users prefer this method for style in 62.7% and structure in 59.5% of comparisons
- **Tradeoff:** Larger training patches transfer more style (texture) but also change more structure, exposing the limits of separation



Park et al., Fig. 6: style/content preference (left) and patch-size tradeoff (right)

Swapping Autoencoder: Principles and Limitations

- **Principle:** Patch co-occurrence gives “texture” / “style” encodings
- **Limitation:** Structure and texture remain coupled: larger training patches can transfer more style and change more structure
- Overall, the spatial/global architecture biases the separation but does not provide exact invariance

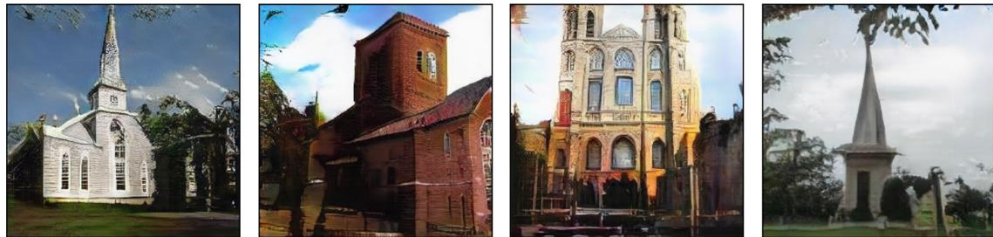
GAN Dissection: CNN Neuron = Semantic Control



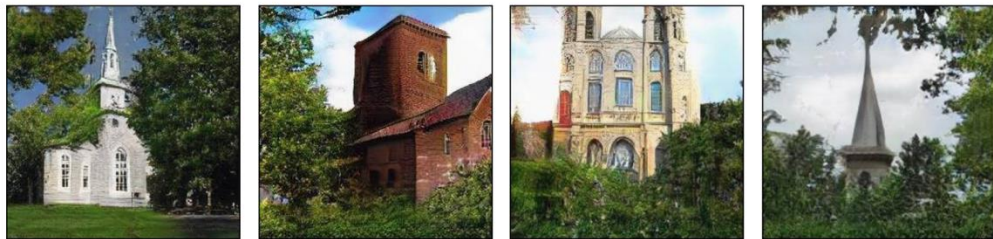
(a) Generate images of churches



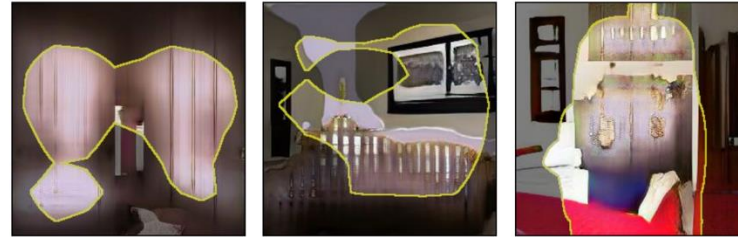
(b) Identify GAN units that match trees



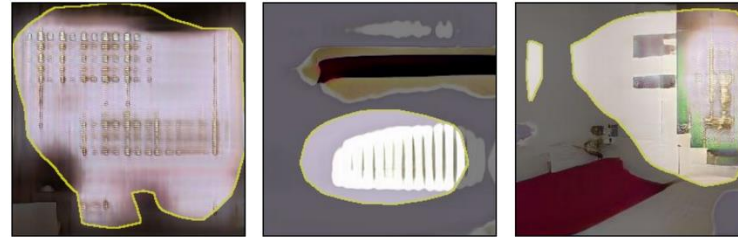
(c) Ablating units removes trees



(d) Activating units adds trees



(e) Identify GAN units that cause artifacts



(f) Bedroom images with artifacts

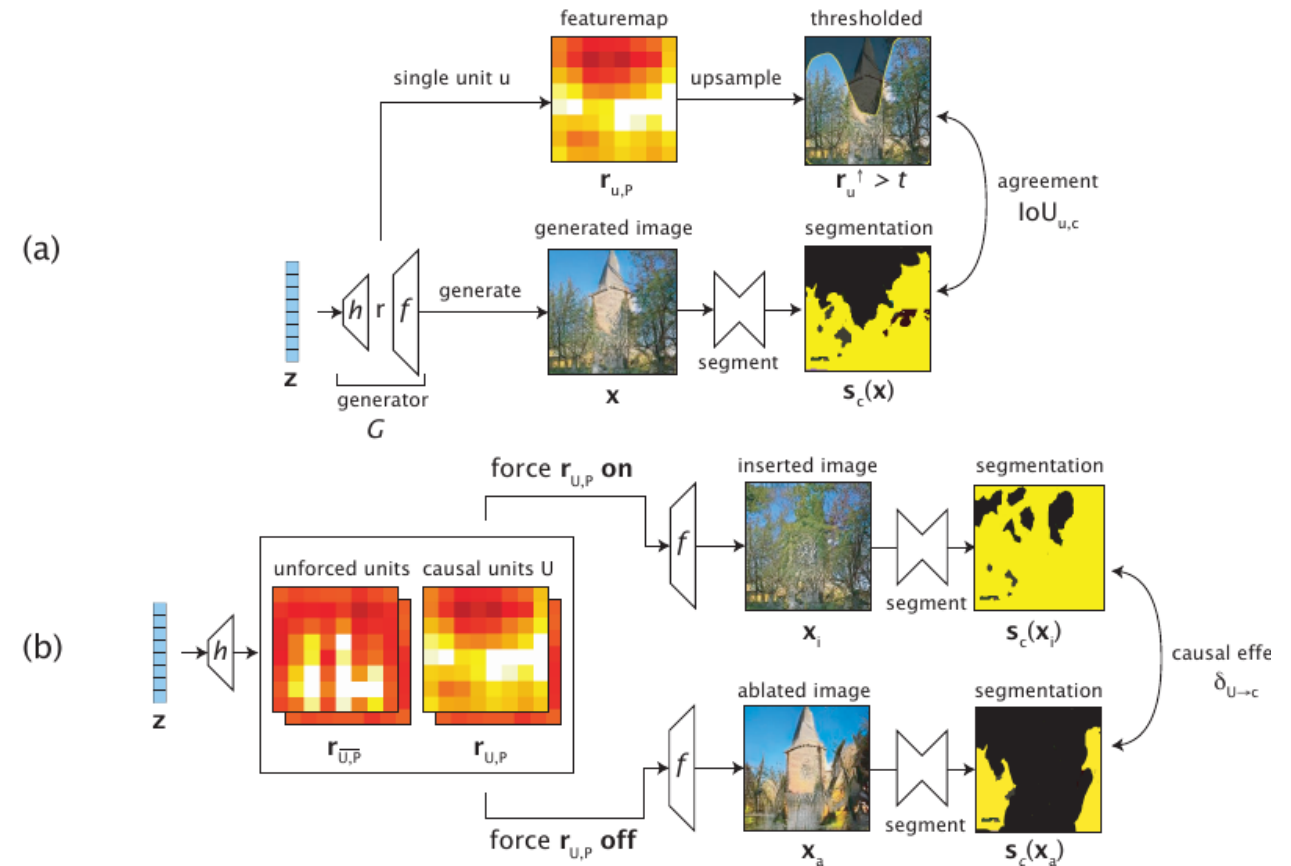


(g) Ablating "artifact" units improves results



GAN Dissection: Correlation into Control

- **Key idea:** A unit is an object control only if intervening on it changes the object
- A pretrained LSUN GAN yields controls for object insertion or removal



GAN Dissection: Principle and Limitations

- **Principle:** Intervene on candidate semantic units to discover object controls
- And distinguish them from correlations
- **Limitation:** The available controls depend on the pretrained GAN and the concepts recognized by the semantic segmenter

Today's Agenda: **Ideas for Control**

1. Learning from examples

- Paired and unpaired label training

2. Leveraging neural network structures

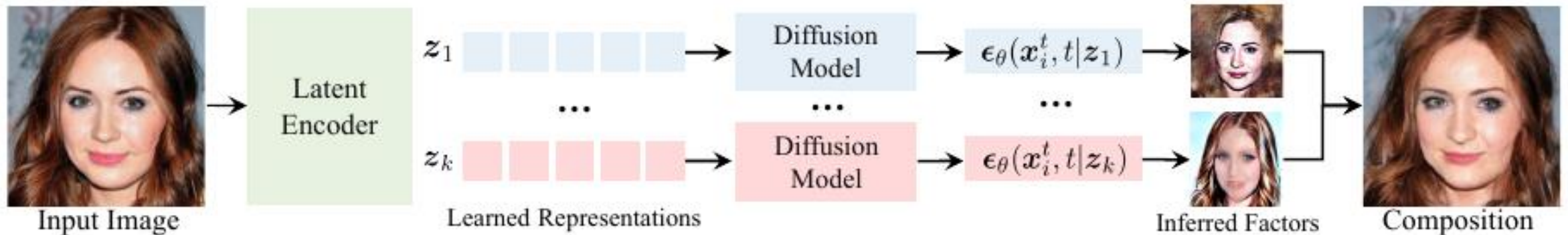
- Patch structures and convolutional structures
- Compositional diffusion structures
- Neural network designs for editing tasks
- Graphics structures

3. Using programs to represent structures

Idea 2.2: Compositional Structures of Diffusions

(De-) Compositional Diffusion Models

- **Key idea:** Explain one image using several compact codes whose denoising contributions combine to reconstruct it
- Learning an ***inference*** model for a diffusion model!

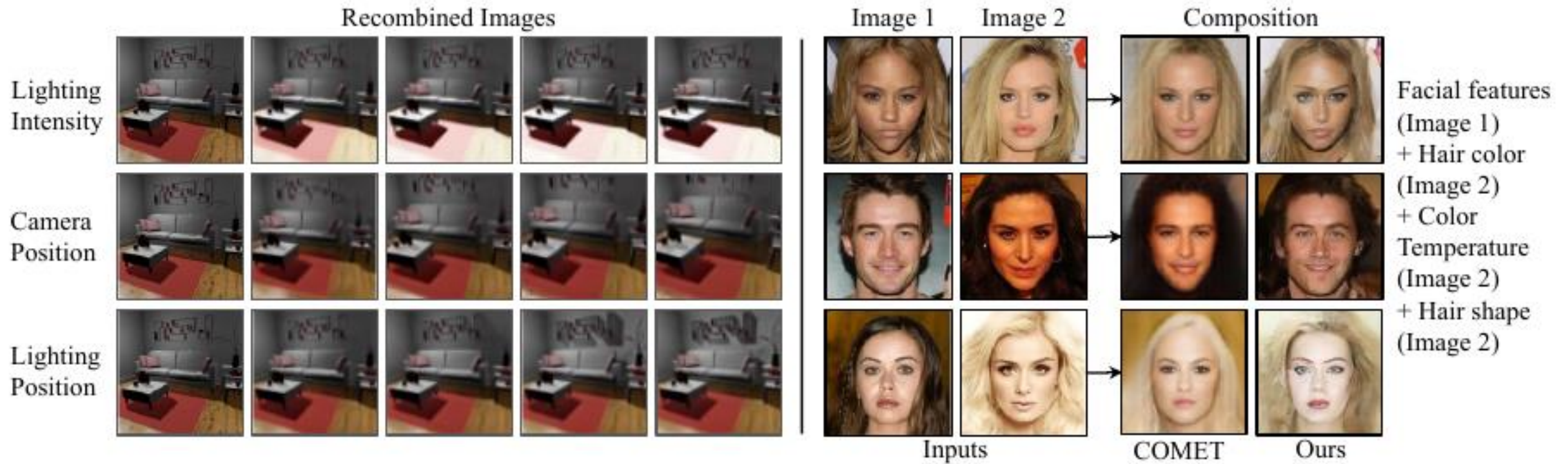


Learning Compositional Factors

- Encoder codes condition denoisers that see the same noisy image and timestep
- The sum of their predictions matches the added noise; networks share parameters
- A low-dimensional bottleneck limits how much each code can store

$$\mathcal{L} = \mathbb{E} \left\| \varepsilon - \sum_k \varepsilon_{\theta}(x_t, t, z_k) \right\|^2$$

Testing Factor Recombination



Compositional Image Decomposition • Fig. 5

Diffusion Decomposition: Principles and Limitations

- Multiple latent factorizations can explain the same reconstructed image
- A texture code may retain shape; one attribute may be spread across codes
- Think about this as a “Latent PCA,” for diffusion models

Today's Agenda: Ideas for Control

1. Learning from examples

- Paired and unpaired label training

2. Leveraging neural network structures

- Patch structures and convolutional structures
- Compositional diffusion structures
- Neural network designs for editing tasks
- Graphics structures

3. Using programs to represent structures

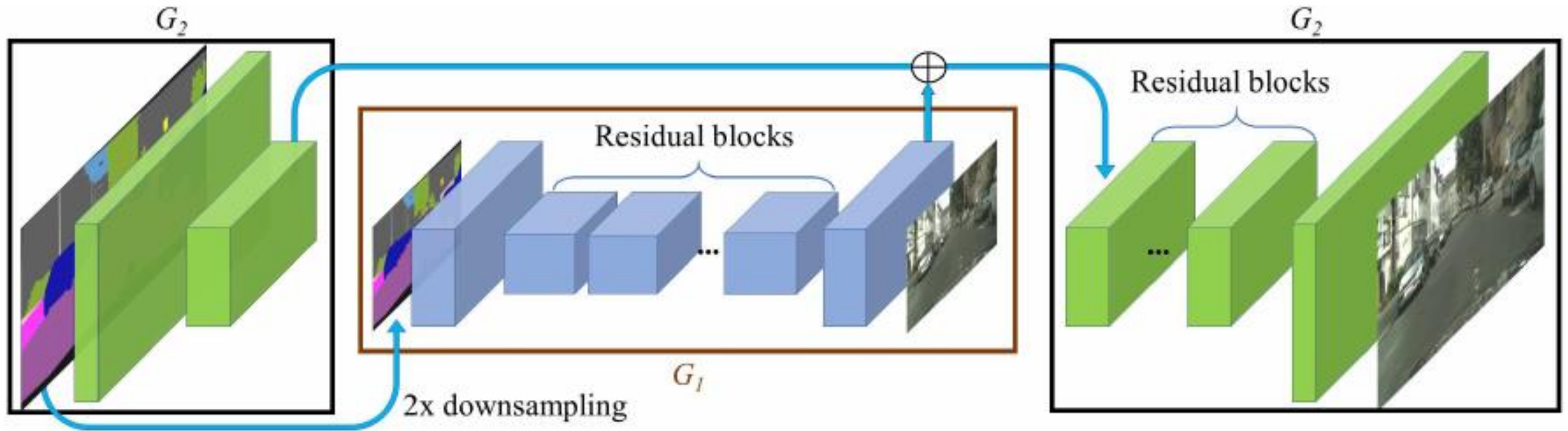
Idea 2.3: Neural Network Designs

1. Course-to-Fine Structures
2. Caveats about Normalization Layers
3. Attention modulation

Structured Conditions for Neural Control

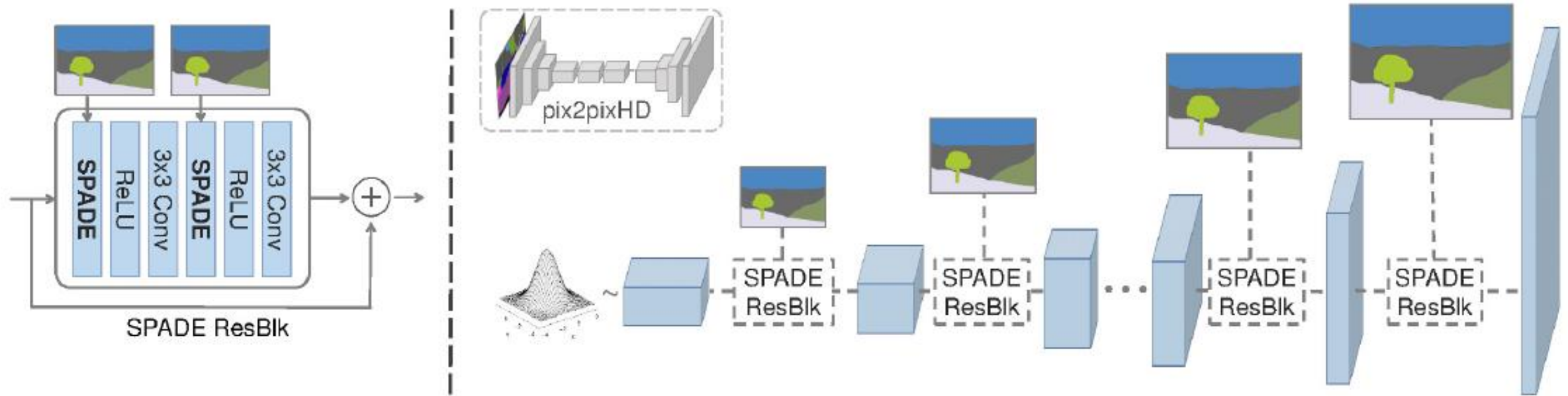
- **Language:** Convenient for semantic changes, but often underspecifies exact geometry or layout
- **Structured input:** A segmentation map, pose, edge map, or depth image specifies spatial intent

pix2pixHD: Control at Multiple Scales

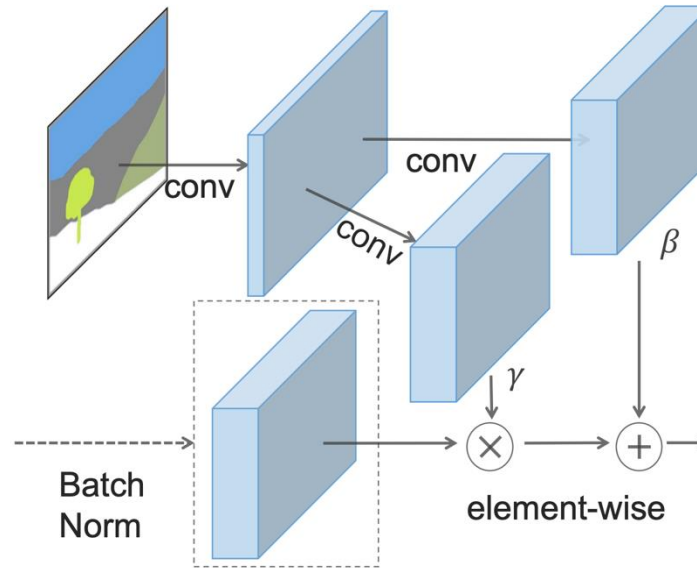


- **Key idea:** Coarse-to-fine generation and multiscale discrimination enable high-resolution label-to-photo synthesis
- **Mechanism:** Label maps and instance edges guide generation, while feature matching stabilizes training

SPADE: Fuse Semantic Information as Affine



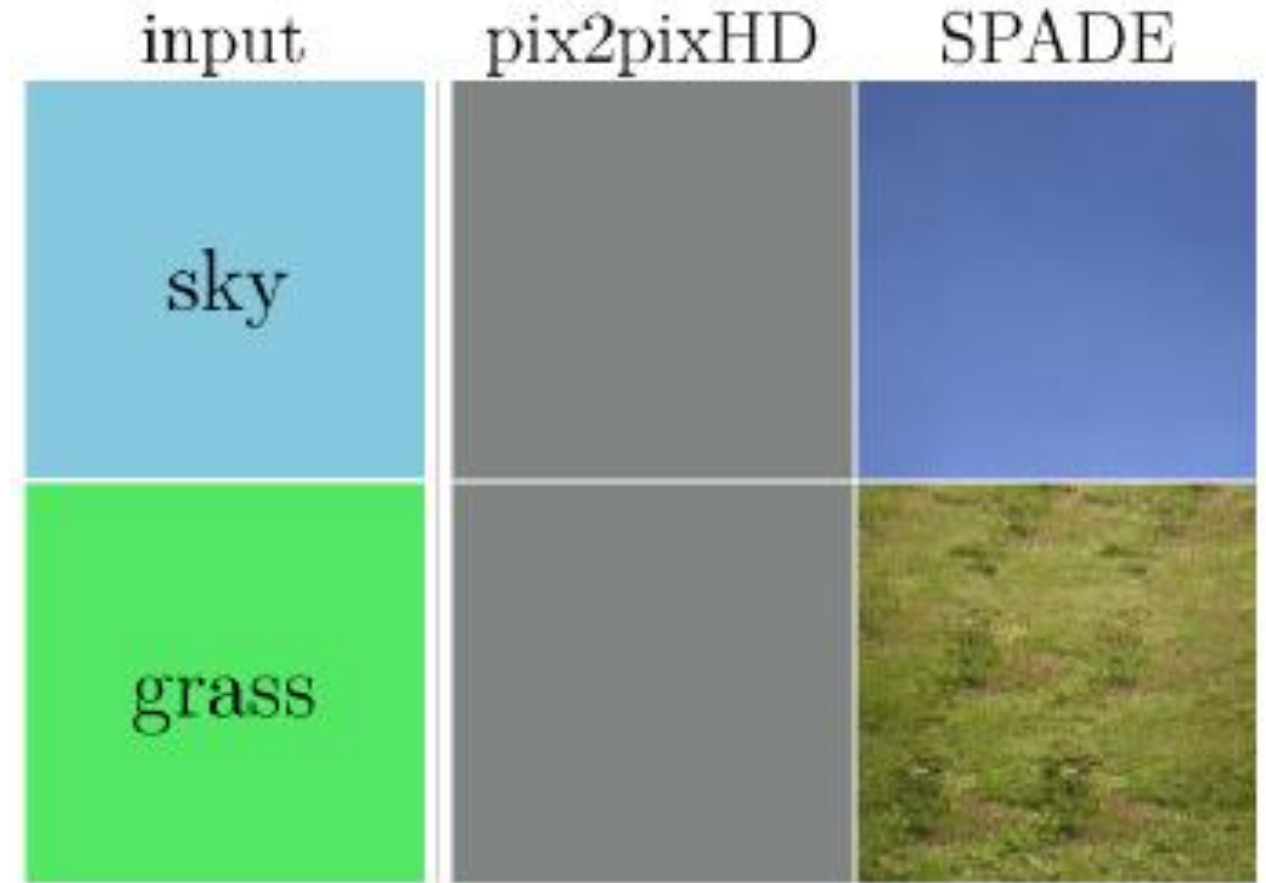
SPADE: Restore Layout after Normalization



- **Key idea:** Restore mask information after normalization using spatially varying scale and bias

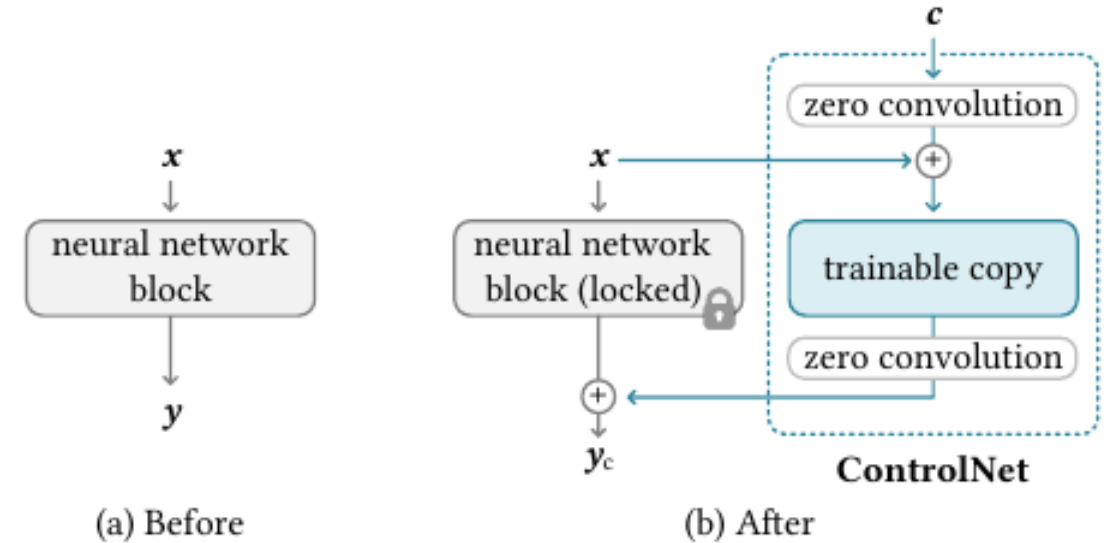
SPADE: Why is this important?

- **Diagnostic:** Uniform masks can lose class identity after normalization!
- Adding back the spatially varying scales and biases



ControlNet: Learn a Residual Controller

- **Problem:** Limited condition-paired data
- **Key idea:** Freeze the pretrained generator and learn control through a copied branch whose residual starts at zero



Zero-Initialized Connections

- **Input:** frozen block features and a condition-aware trainable branch
- At initialization, the outer zero convolution makes the residual exactly zero

$$y_c = F(x) + Z(H(x, c)), \quad Z = 0 \Rightarrow y_c = F(x)$$

The outer weights receive gradients first. Copied features then adapt

Training with Structured Conditions

- **Examples:** Canny edges, estimated depth, pose keypoints, or segmentation labels
- **Target:** predict diffusion noise, conditioned on the map and text

ControlNet: Control without Sacrificing Quality

- **Key question:** Does the added branch improve spatial adherence while retaining generative quality?
- **Segmentation evaluation:** segment generated images and compare with the input labels
- **Condition reconstruction IoU $\uparrow$:** ControlNet 0.35; ControlNet-lite 0.32
- **Quality FID $\downarrow$:** 15.27 vs. 17.92 for the lite model
- **Sketch user study:** rank quality and condition fidelity separately, on a 1–5 scale

ControlNet: Principles and Limitations

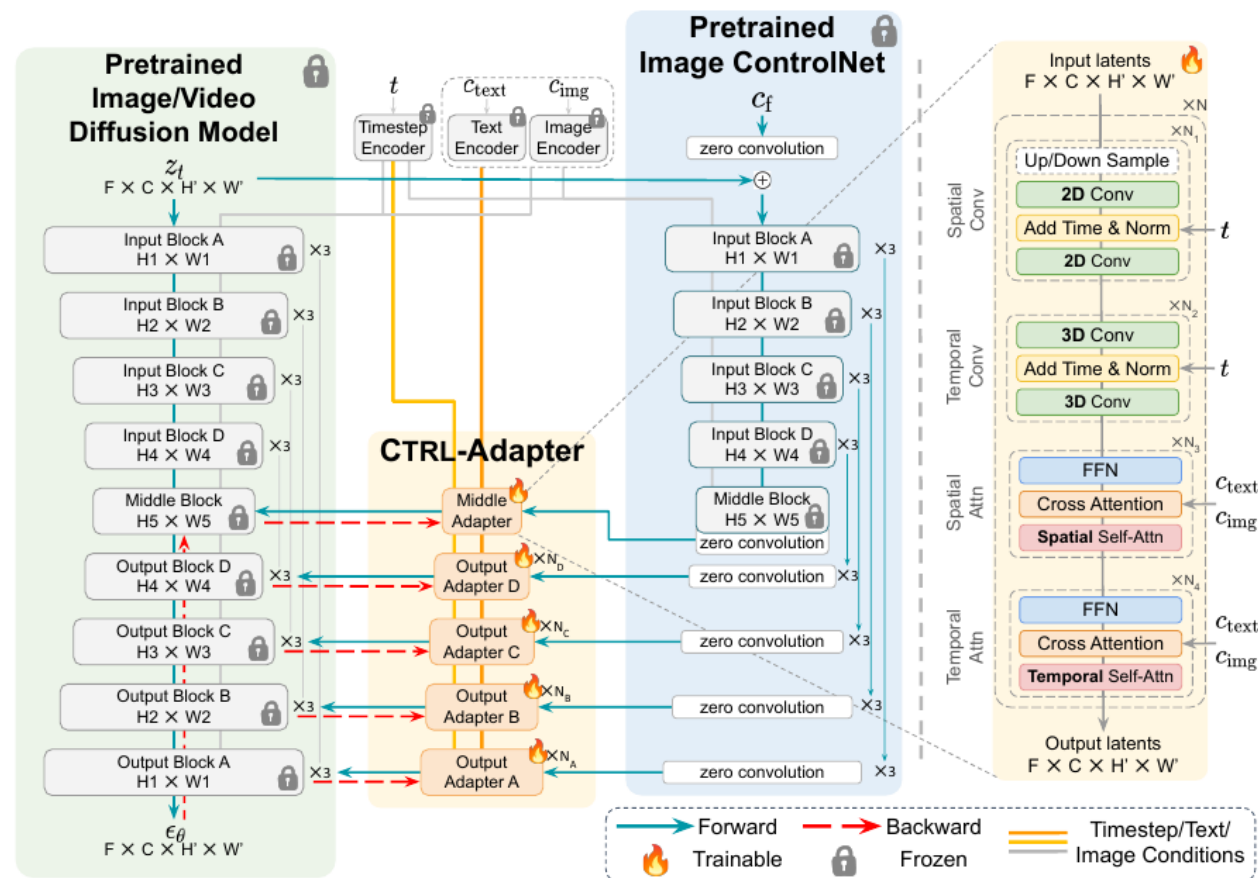
- **Principle:** A zero-initialized residual branch learns spatial conditioning while starting from a frozen generator

Extensions to ControlNets

- **Ctrl-Adapter:** map existing ControlNet features into new image/video backbones
- **ControlNet++:** add condition reconstruction feedback to denoising training
- **Instruct-Nerf2Nerf:** lifting 2D edits to 3D

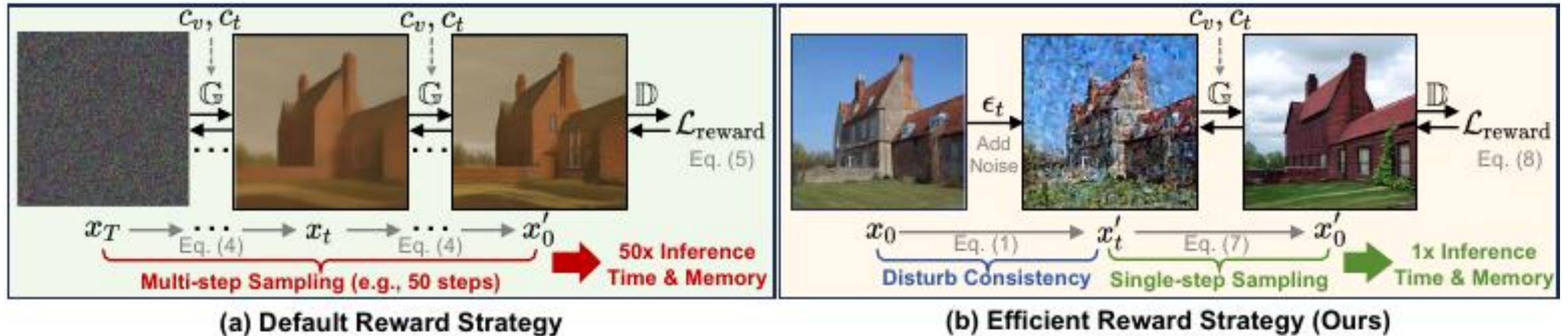
Ctrl-Adapter: Transfer an Existing ControlNet

- **Problem:** A ControlNet trained for one backbone cannot directly control a new generator
- **Key idea:** Learn an interface between frozen ControlNet features and a frozen target backbone

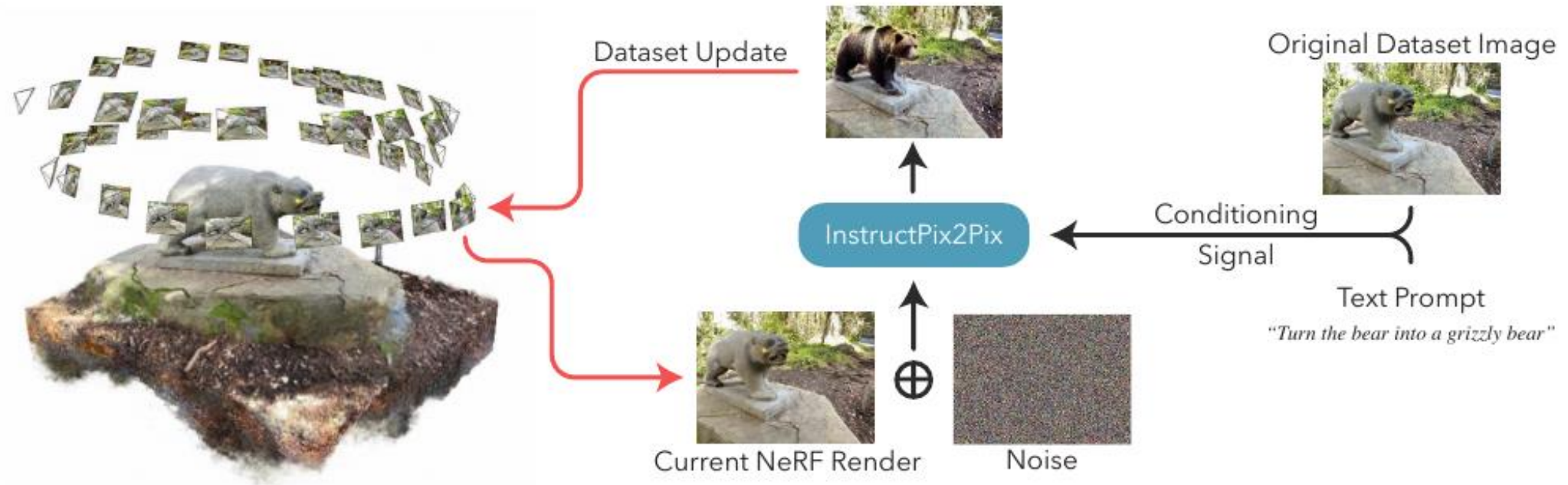


ControlNet++: Train on the Control Error

- **Problem:** Denoising loss does not directly measure whether the output obeys the requested spatial condition
- **Key idea:** Predict the condition from the generated estimate and penalize disagreement with the input map
- They also have additional tricks to make training more efficient (partial denoising)



Instruct-NeRF2NeRF: Edit the Training Targets



- **Key idea:** NeRF learns a rendering function of a scene from a collection of images
- **Mechanism:** Use the source NeRF to generate images from different views; Edit rendered views with InstructPix2Pix; replace training targets

Today's Agenda: **Ideas for Control**

1. Learning from examples

- Paired and unpaired label training

2. Leveraging neural network structures

- Patch structures and convolutional structures
- Compositional diffusion structures
- Neural network designs for editing tasks
- Graphics structures

3. Using programs to represent structures

Graphics Structures

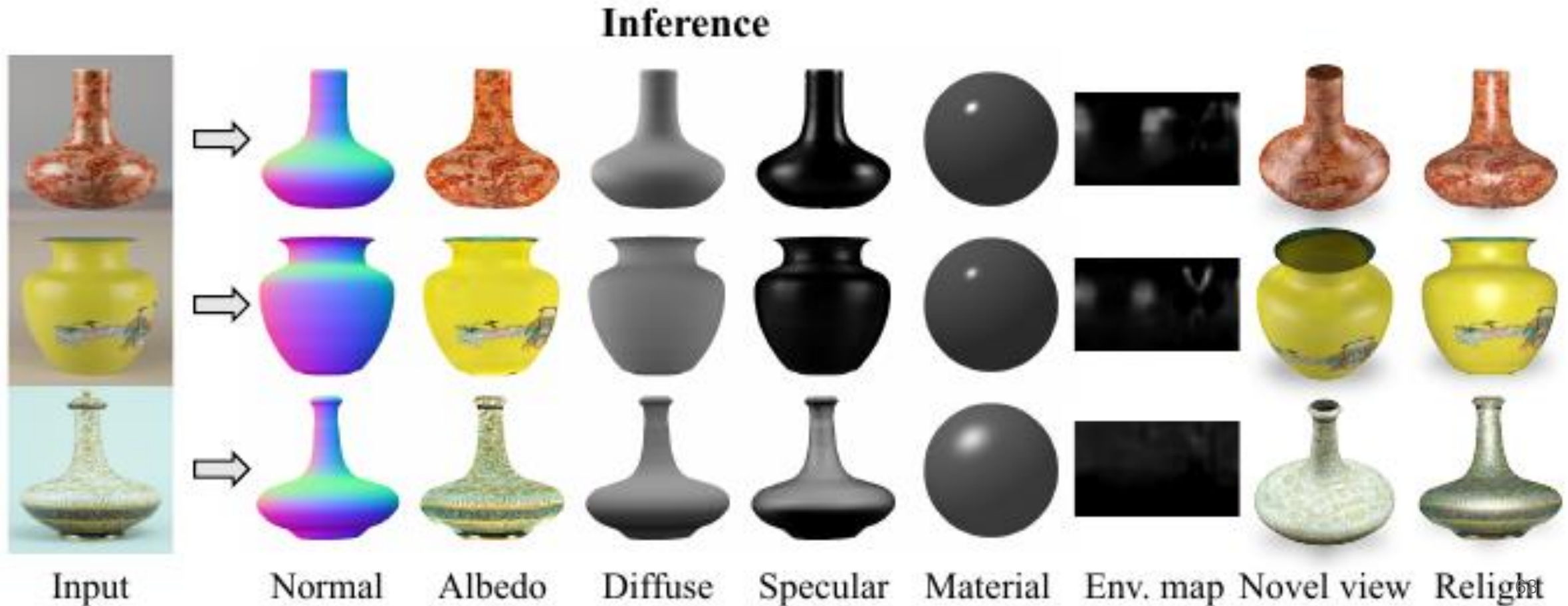
- **RADAR:** Infer factors using a rendering equation and learned priors
- **IC-Light:** Train a diffusion model with consistency constraints from light transport

A related structural approach: constrained image formation

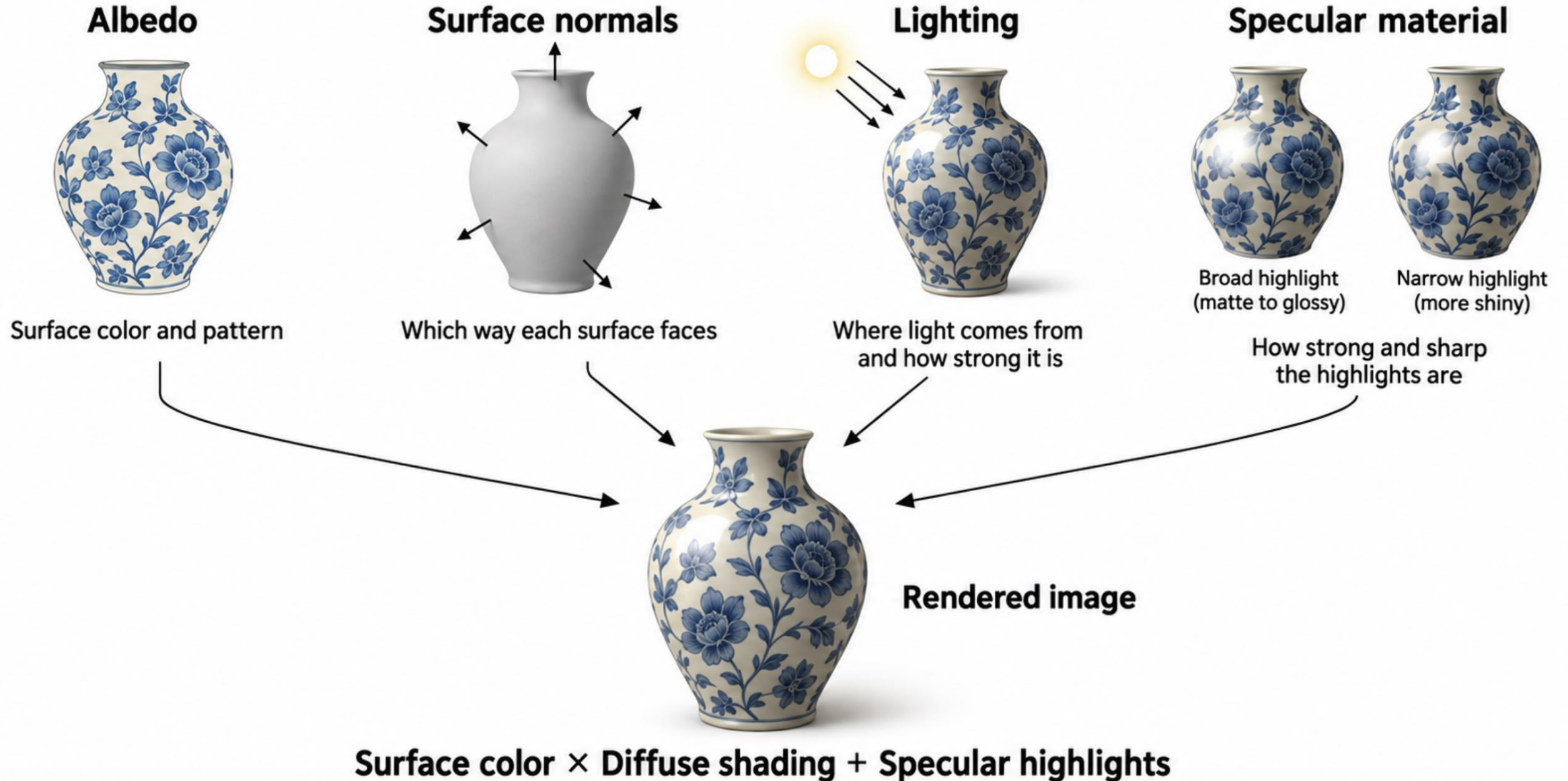
RADAR: Leverage the Rendering Equations

Wu et al., CVPR 2021 · Fig. 1

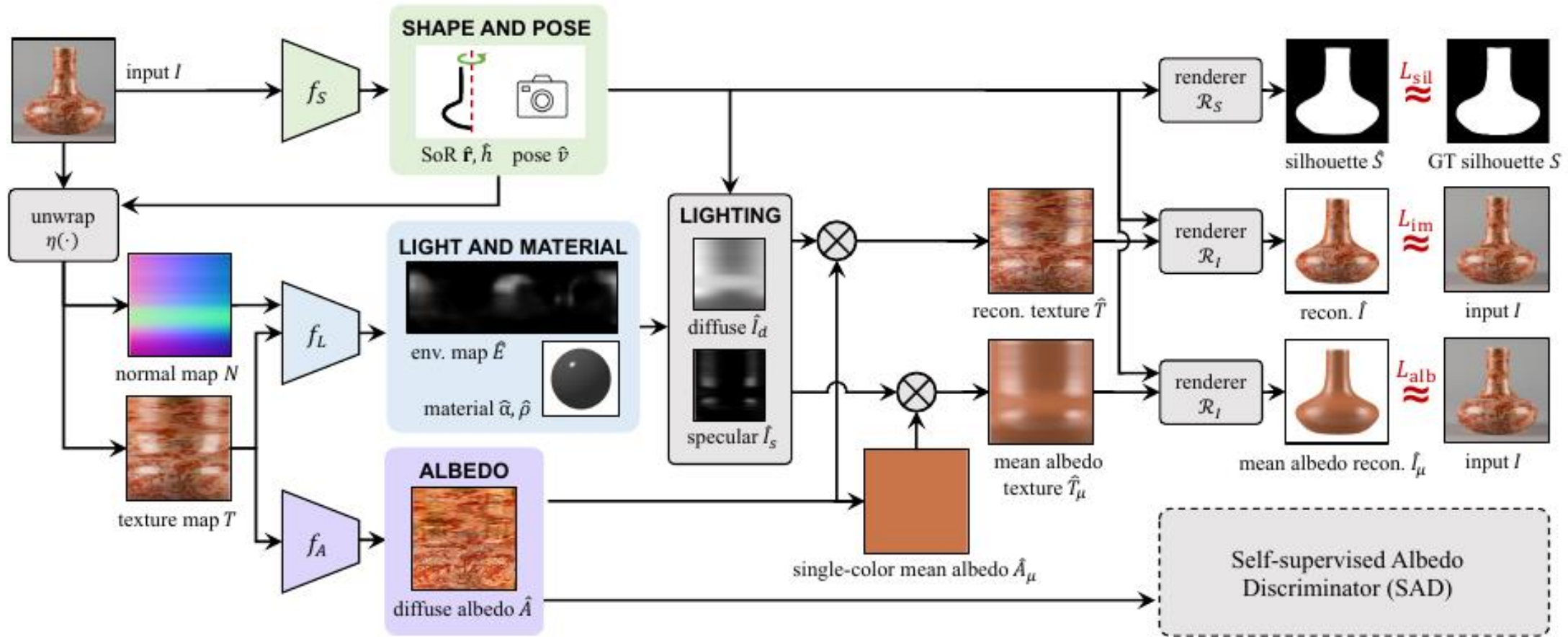
- **Input:** a single vase image; silhouettes provide training supervision
- **Output:** shape/pose, albedo, specular material, and environment lighting



RADAR: Leverage the Rendering Equations



RADAR: Leverage the Rendering Equations



- **Key idea:** Use a shape prior and physical rendering priors

RADAR: Leverage the Rendering Equations

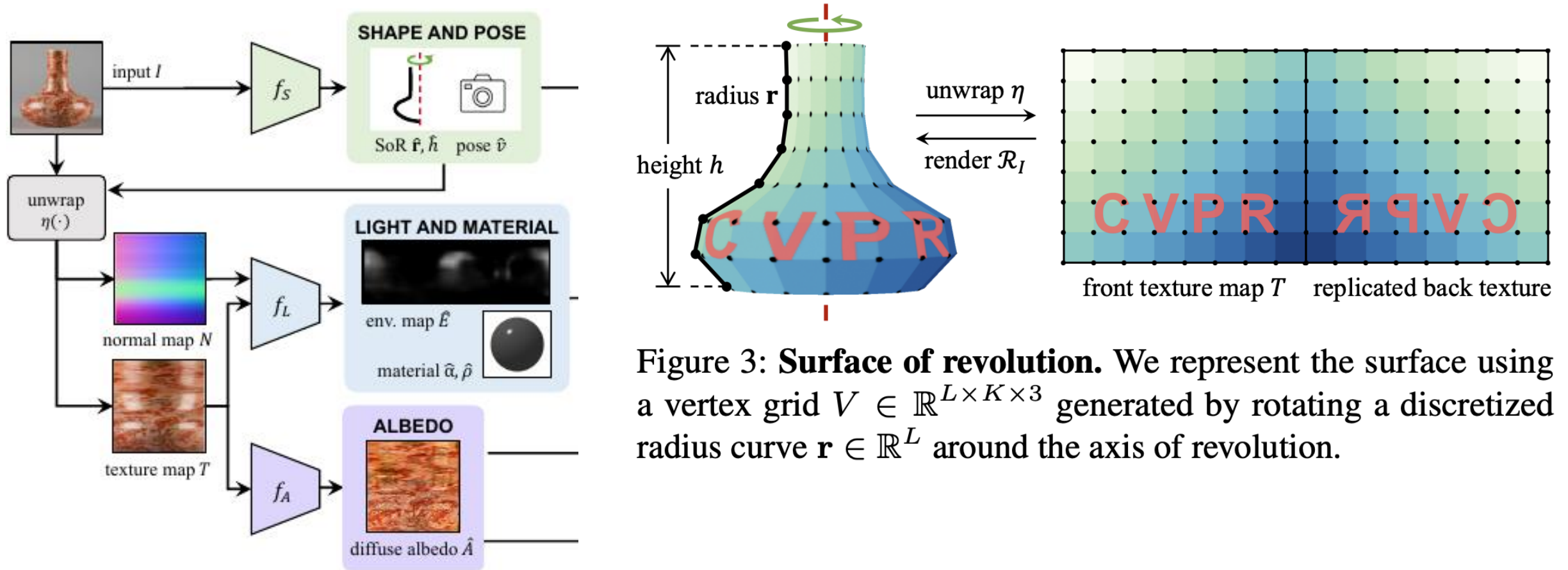


Figure 3: **Surface of revolution.** We represent the surface using a vertex grid $V \in \mathbb{R}^{L \times K \times 3}$ generated by rotating a discretized radius curve $\mathbf{r} \in \mathbb{R}^L$ around the axis of revolution.

- **Key idea:** Use a shape prior and physical rendering priors

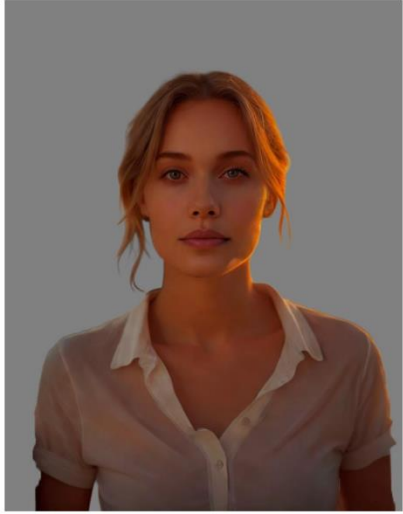
RADAR: Neural Network Priors

- **Objective:** silhouette/appearance reconstruction plus albedo and shading constraints
- Discourages highlights being stored as paint: single-color albedo rendering

RADAR: Principles and Limitations

- **Principle:** Constrain inverse rendering with a shape prior and an image-formation model to separate material and light
- **Limitation:** High-contrast paint can be mistaken for specularity, and complex lighting can be misestimated

IC-Light: Leverage Properties of the Rendering Equations



input



“... sunlight through the blinds, near window blinds”



input



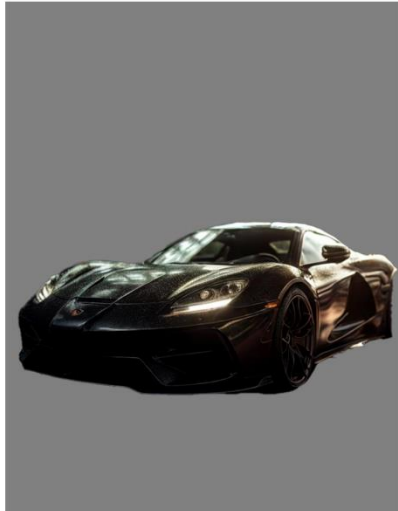
“... sunlight from the left side, beach”



input



“... magic golden lit, forest”



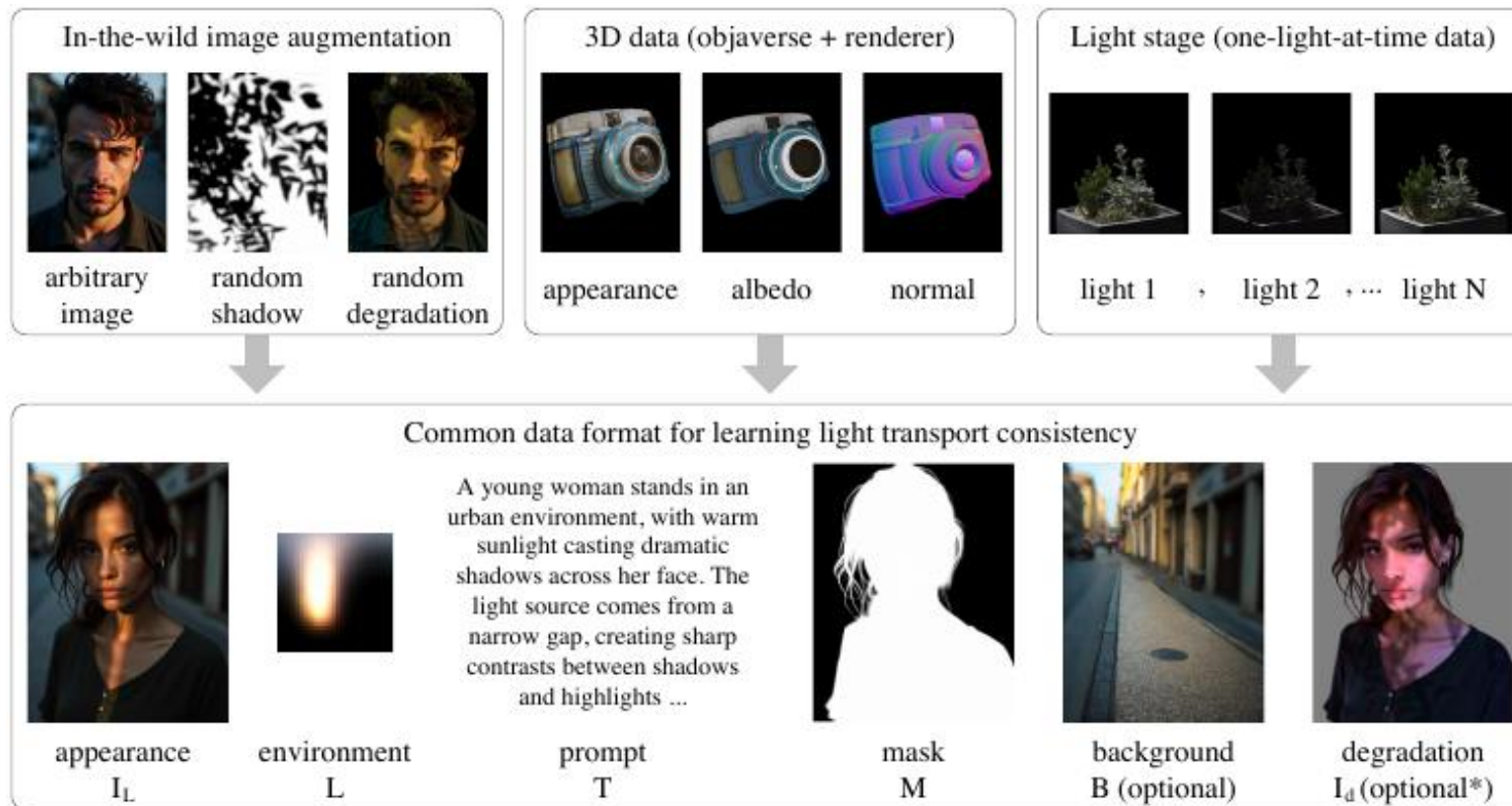
input



“... neo punk, city night”

IC-Light: Diverse Relighting Data

- **Input:** degraded appearance, lighting, text, mask, and optional background
- **Output:** a relit appearance aligned with the target illumination
- Training mixes 6M wild images, 4M rendered examples, and light-stage data



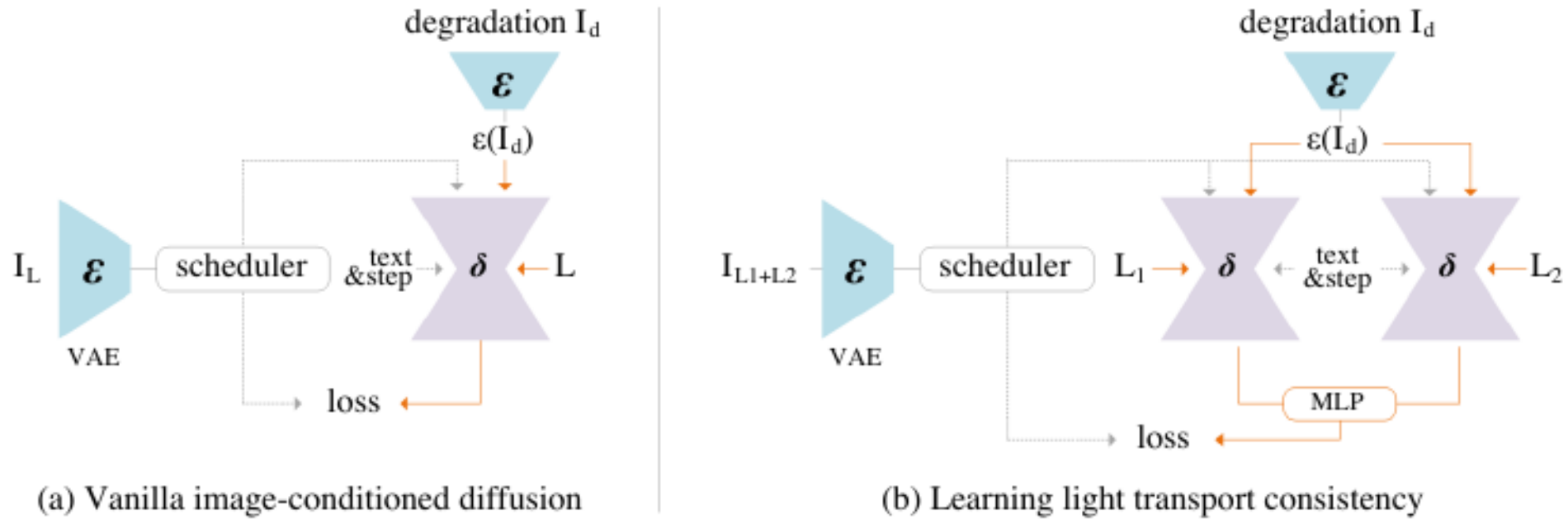
Zhang, Rao & Agrawala, ICLR 2025 · Fig. 2

Linearity of Light Transport

- **Input:** illumination (environment map)
- **Output:** scene radiance under a fixed transport operator
- **Assumptions:** linear radiance response

$$I_{L_1+L_2}^* = T(L_1 + L_2) = I_{L_1}^* + I_{L_2}^*$$

IC-Light: Additivity as an Auxiliary Supervision



- **Mechanism:** Use an adapter for nonlinear compositions (e.g., LDR)

Learning Consistent Light Transport

- Split an environment map into complementary lights; share the noisy input
- Predict each part's appearance and compare their adapted combination to the target
- Optimize the diffusion target loss plus a weighted consistency term

$$\mathcal{L}_{\text{consistency}} = \|M \odot (\varepsilon_{L_1+L_2} - \varphi(\varepsilon_{L_1}, \varepsilon_{L_2}))\|^2$$

Today's Agenda: Ideas for Control

1. Learning from examples

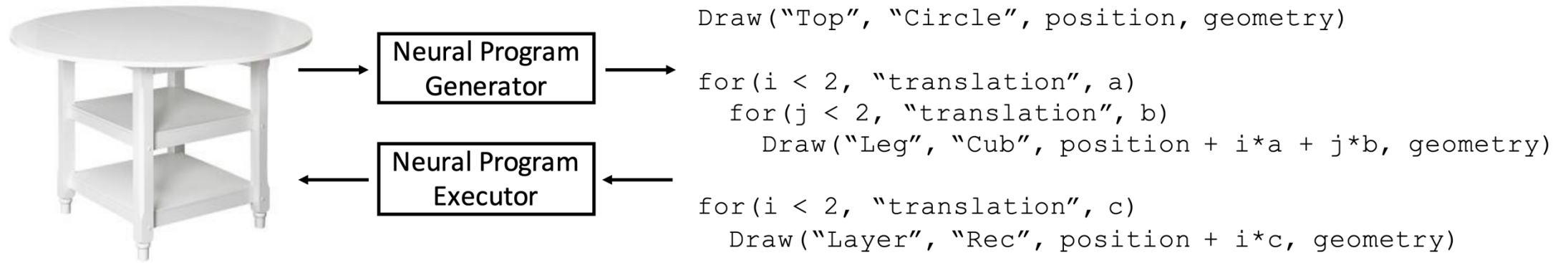
- Paired and unpaired label training

2. Leveraging neural network structures

- Patch structures and convolutional structures
- Compositional diffusion structures
- Neural network designs for editing tasks
- Graphics structures

3. Using programs to represent structures

Idea: Use Programs to Represent 3D Structures



Edit Interfaces for 3D Shape Programs

- **Input:** Furniture programs with cuboids, attachments, and symmetry operations
- **Edit → output:** Change parameters; execute the same structure to produce a related shape
- **Example:** Shared seat-width parameters can move attached legs coherently



ShapeAssembly: Generate Relationships

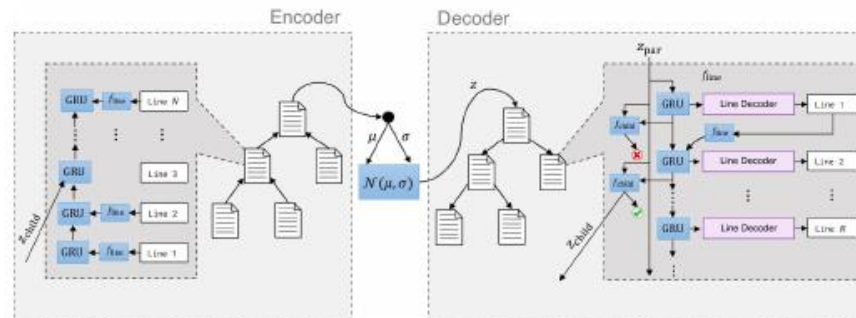
ShapeAssembly DSL (Section 4)

Start $\rightarrow$ BBlock; CBlock; ABlock; SBlock;
 BBlock $\rightarrow$ bbox = Cuboid(l, h, w, True)
 CBlock $\rightarrow$ $c_n = \text{Cuboid}(l, w, h, a)$; CBlock | None
 ABlock $\rightarrow$ Attach; ABlock | Squeeze; ABlock | None
 SBlock $\rightarrow$ Reflect; SBlock | Translate; SBlock | None
 Attach $\rightarrow$ attach($c_{n1}, c_{n2}, x_1, y_1, z_1, x_2, y_2, z_2$)
 Squeeze $\rightarrow$ squeeze($c_{n1}, c_{n2}, c_{n3}, f, u, v$)
 Reflect $\rightarrow$ reflect(c_{n1}, axis)
 Translate $\rightarrow$ translate($c_{n1}, \text{axis}, m, d$)
 $f \rightarrow$ right | left | top | bot | front | back
 $\text{axis} \rightarrow$ X | Y | Z
 $l, h, w \in \mathbb{R}^3$
 $x, y, z, u, v, d \in [0, 1]^2$
 $a \in [\text{True}, \text{False}]$
 $n, m \in \mathbb{Z}^+$

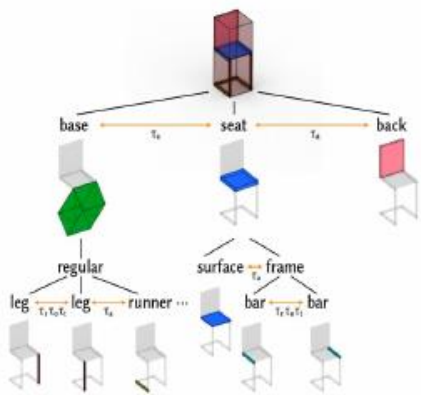
Shapes to Training Programs (Section 5)



Learning to Generate Programs (Section 6)



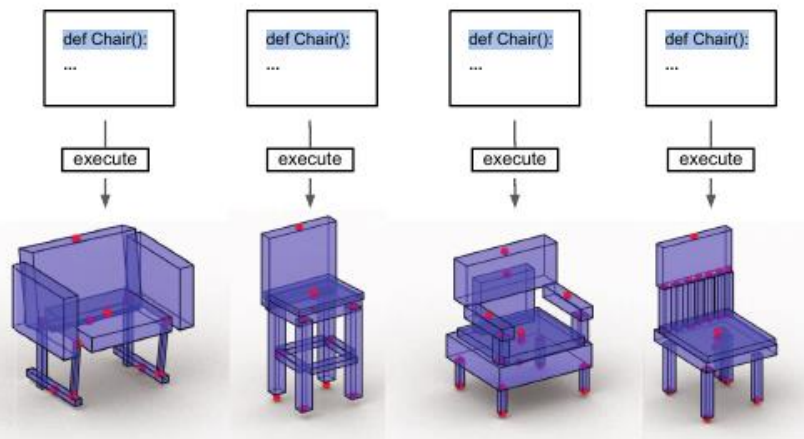
Input Hierarchical Part Graphs



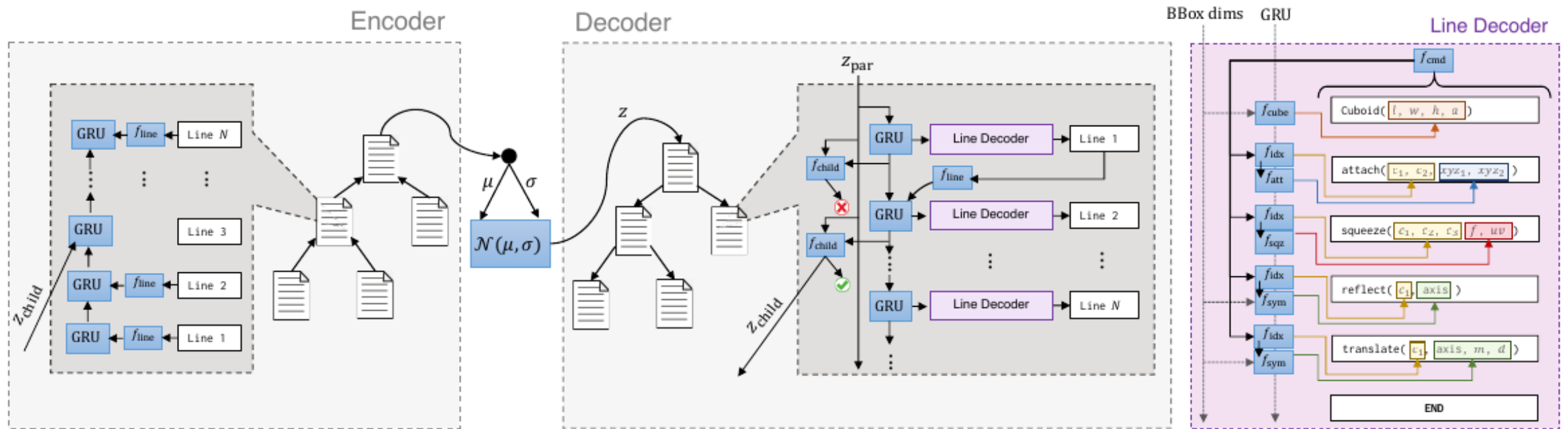
```

def Chair():
    bbox = Cuboid(.7, 1.7, .5, True)
    prog1 = Program1(.7, .6, .5, True)
    prog2 = Program2(.7, .9, .05, True)
    cube2 = Cuboid(.7, .15, .5, True)
    attach(prog1, bbox, .5, 0, .5, .5, 0, .5)
    attach(cube2, prog1, .5, 0, .5, .5, 1, .5)
    squeeze(prog2, bbox, cube2, top, .5, .1)

def Program1(l, w, h, aligned):
    bbox = Cuboid(.7, .6, .5, True)
    prog3 = Program3(.05, .6, .5, True)
    squeeze(prog3, bbox, bbox, top, 0, .5)
    reflect(prog3, X)
    ...
    
```



ShapeAssembly: Learn to Write the Program



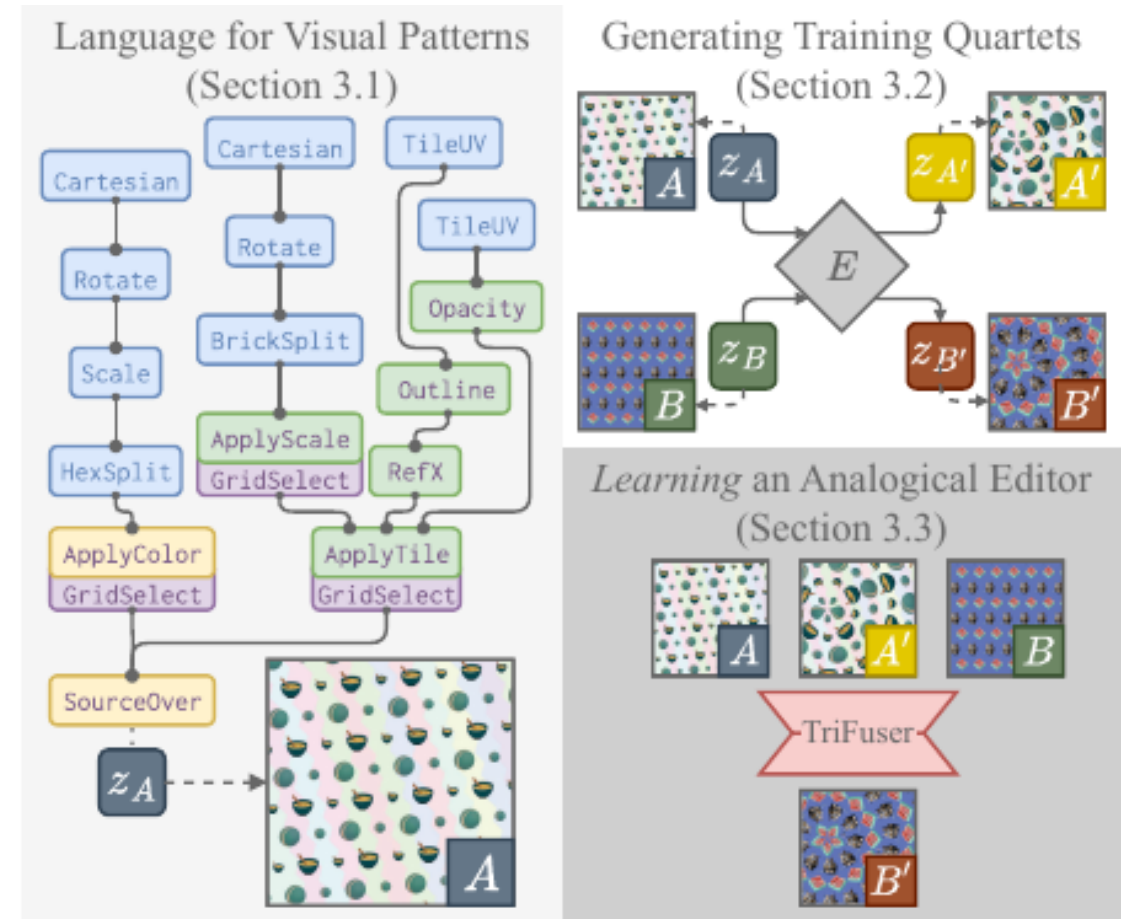
- **Model:** A hierarchical VAE recursively encodes and decodes program commands

ShapeAssembly: Principles and Limitations

- **Principle:** Generate executable attachment and symmetry relationships so part edits propagate through an assembly
- **Limitation:** Cuboid programs abstract away detailed surfaces, and program extraction excludes some valid shapes

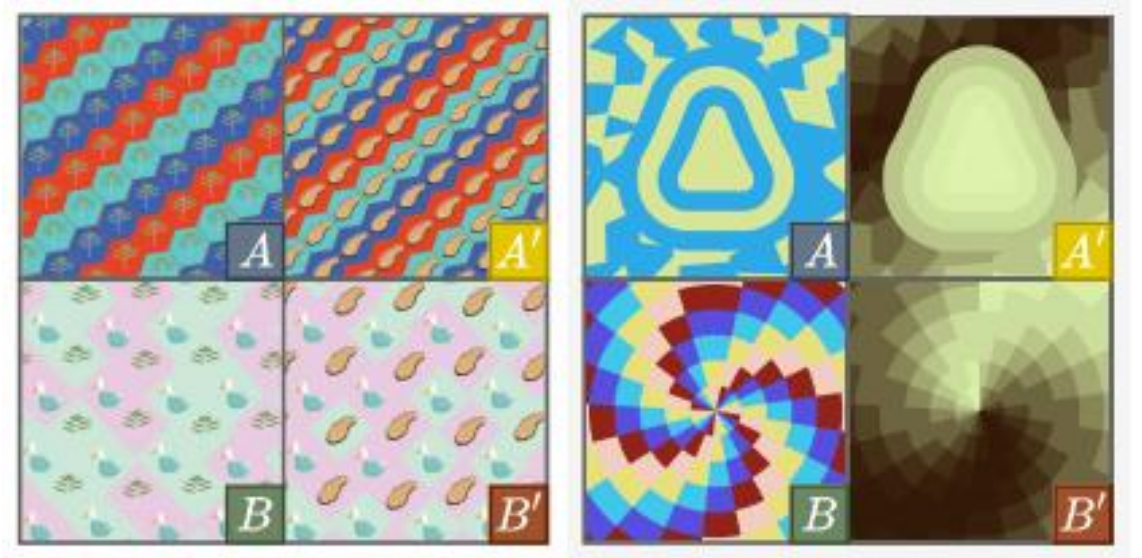
Pattern Programs

- **Problem:** Learning to do “analogy-based” editing
- **Key idea:** Apply one program operation twice on different images to construct a labeled quartet
- **Data:** 1M procedural quartets



Program-Generated Paired Paired Data

- **Problem:** Learning to do “analogy-based” editing
- **Key idea:** Apply one program operation twice on different images to construct a labeled quartet
- **Data:** 1M procedural quartets



Motion Programs

Text prompt

<svg> ... </svg>
Move the orange circle above the rectangular shape.
In the meantime, rotate the letter H clockwise by 90 degrees.

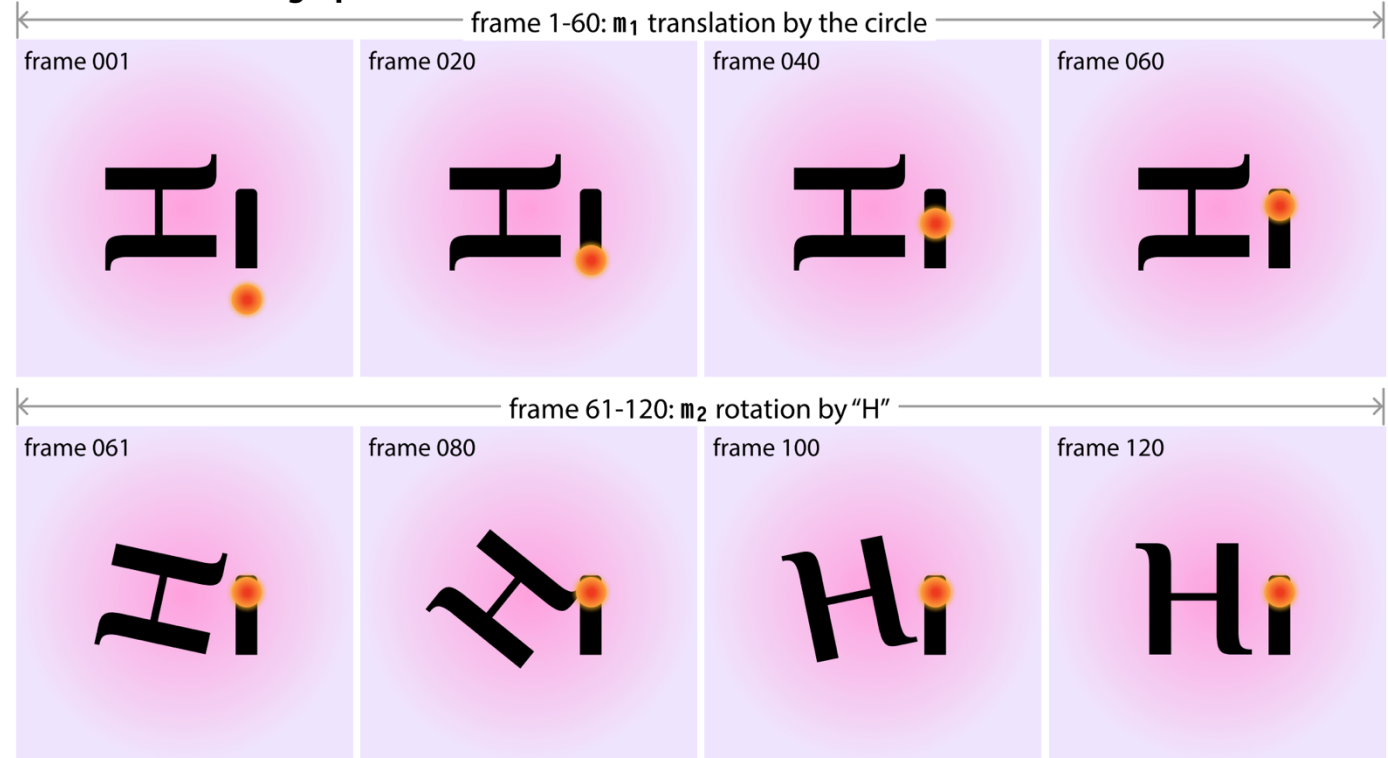
MoVer program

```
o1 = 1o.clr(o,"orange")^shp(o,"circle") # check for an orange circle
o2 = 1o.shp(o,"rectangle")               # check for a rectangle
o3 = 1o.id(o,"H")                         # check for the id "H"
# check for a translation by the circle to the top of the rectangle
m1 = 1m.type(m,"trn")^agt(m,o1)^post(m,top(o1,o2))
# check for a clockwise rotation of 90 degrees by "H"
m2 = 1m.type(m,"rot")^agt(m,o3)^dir(m,"cw")^mag(m,90)
# assert that the translation and the rotation overlap in time
while(m1,m2)
```

MoVer verification report

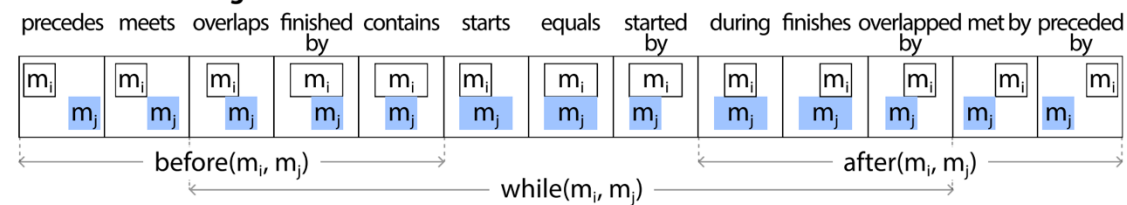
m1 = 1m.type(m,"trn")^agt(m,o1)^post(m,top(o1,o2))	false
top(o1,o2)	false
post(m,top(o1,o2))	false
m2 = 1m.type(m,"rot")^agt(m,o3)^dir(m,"cw")^mag(m,90)	true
while(m1,m2)	false

Generated motion graphics animation

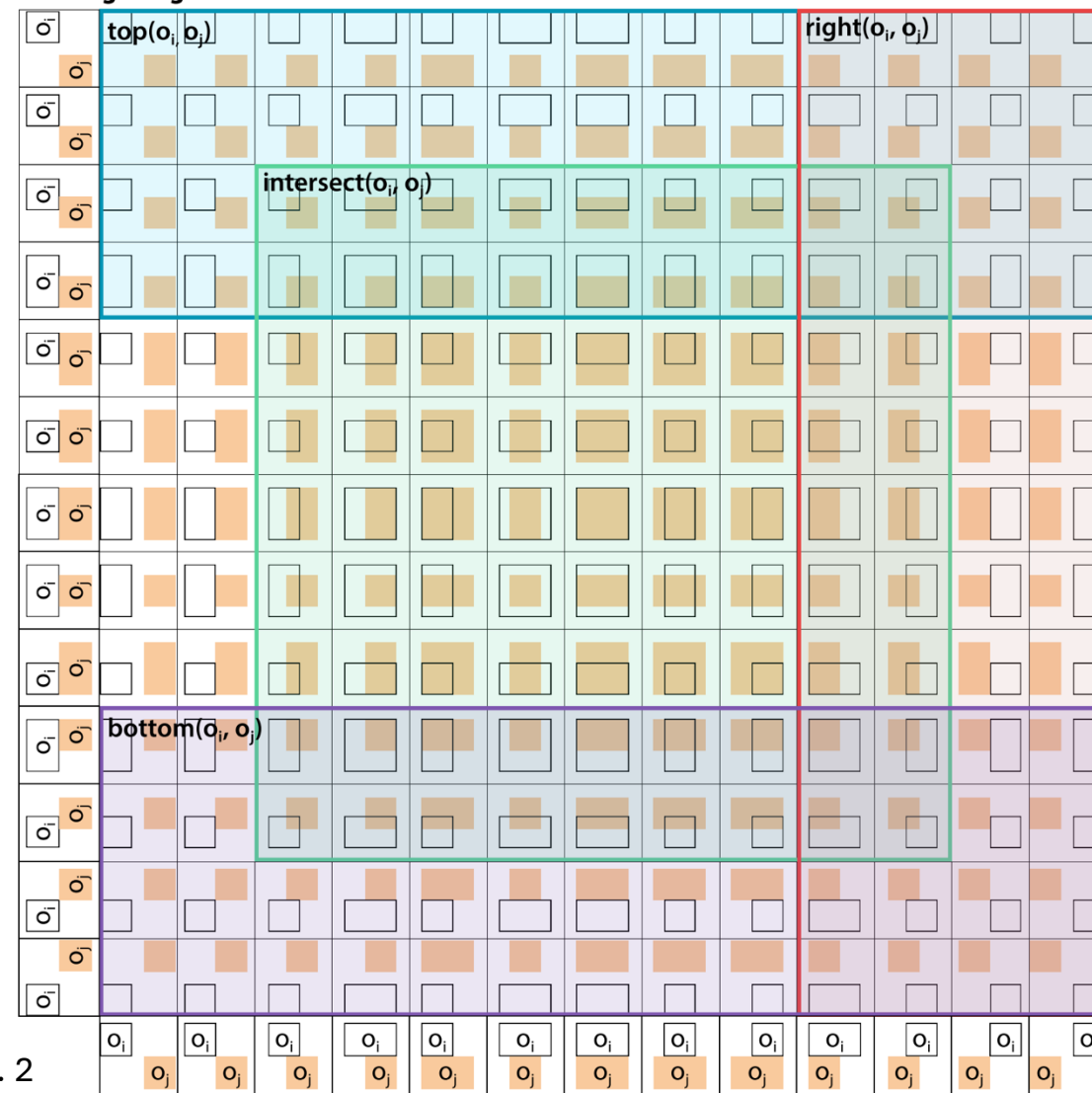


Motion Programs: Qualitative Predicates

Allen's Interval Algebra Predicates



Rectangle Algebra Predicates

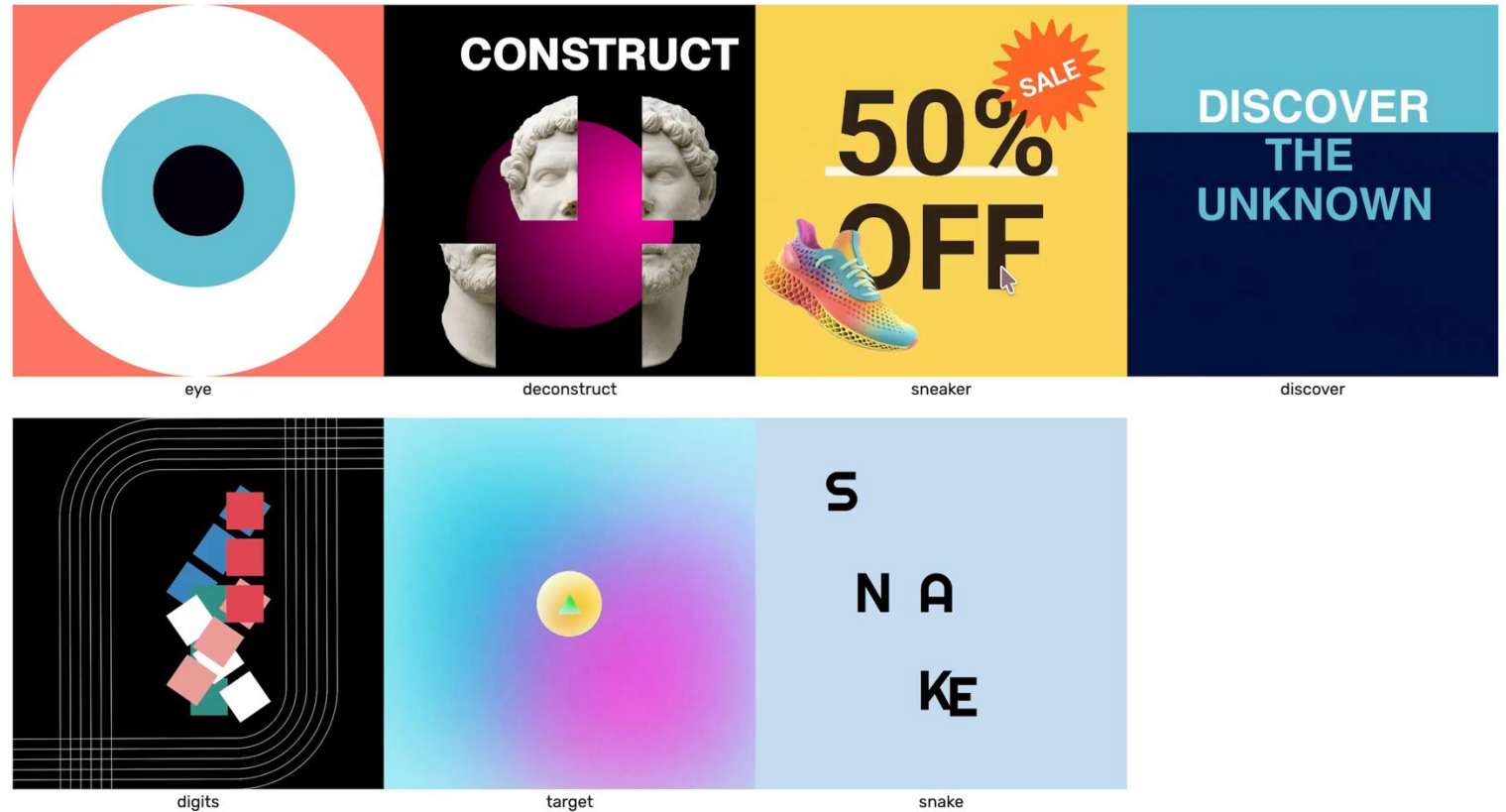


Motion Programs

Figures in paper



More fully-automated examples



Questions for Discussion

- What are the tradeoffs between text-based controls and structured controls (in your domain of interest)?
- What are the tradeoffs between program-generated examples and generative-model—generated examples?
- Structures are great. But how can we trade-off human-defined structures (interpretable not expressive) and algorithm-discovered structures (expressive but not interpretable)?